

UNIVERSITÀ
DEGLI STUDI
DI PADOVA



DIPARTIMENTO
DI INGEGNERIA
DELL'INFORMAZIONE

DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE

CORSO DI LAUREA IN INGEGNERIA INFORMATICA

Calcolo della mediana con memoria limitata: studio e sperimentazione dell'algoritmo multi-pass di Munro e Paterson.

Relatore

Prof. Andrea Pietracaprina

Laureando

Alessandro Barbacetto

ANNO ACCADEMICO 2025–2026

Data di laurea: 11 Novembre 2026

Indice

1 Introduzione **1**

1.1 Il problema della selezione e della mediana 1

1.2 Cenni storici 1

1.3 Obiettivi e struttura della tesi 2

2 Problema della mediana e algoritmo SELECT **4**

2.1 Definizione del problema e notazione 4

2.2 Selezione mediante pivot 5

2.3 Algoritmo SELECT: median of medians 7

2.4 Pseudocodice e complessità 8

2.5 Limiti del modello classico 9

3 Selezione con memoria limitata **11**

3.1 Modello computazionale e notazione 11

3.2 Struttura generale dell'algoritmo 13

3.3 Struttura di una passata 15

3.3.1 Costruzione del campione 15

3.3.2 Verifica dei vincoli della memoria 18

3.3.3 Scelta della nuova coppia di filtri 18

3.4 Pseudocodice dell'algoritmo 21

3.4.1 Realizzazione del campione 22

3.4.2 Singola passata 23

3.4.3 Algoritmo multi-pass completo 25

3.5 Correttezza e terminazione 27

4 Implementazione e analisi sperimentale **29**

4.1 Scelte implementative 29

4.2 Realizzazione dell'algoritmo 30

4.3 Metodologia sperimentale 31

4.4 Memoria e numero di passate 32

4.5 Candidati effettivi e limite teorico 36

4.6 Considerazioni finali e limiti 40

5 Esempio applicativo **42**

5.1 Scenario applicativo 42
5.2 Modello semplificato 44
5.3 Finestra termica di riferimento 45
5.4 Individuazione dei giri di preparazione 47
5.5 Possibile estensione a quattro pneumatici 47

Bibliografia **51**

Capitolo 1

Introduzione

1.1 Il problema della selezione e della mediana

Il problema della selezione consiste nell'individuare, all'interno di un insieme o di una sequenza di elementi provenienti da un dominio totalmente ordinato, l'elemento che occuperebbe una determinata posizione se i dati fossero ordinati. Assegnato dunque un rango k, che verrà definito più precisamente in seguito, l'obiettivo è determinare il k-esimo elemento senza ricorrere necessariamente all'ordinamento completo dell'input, che costituirebbe una soluzione immediata ma non necessaria al problema.

Un caso particolare del problema della selezione è il calcolo della mediana. Una volta ordinato l'input, la mediana corrisponde infatti a un elemento di rango centrale e può quindi essere ottenuta come un particolare caso del problema di selezione di un elemento di rango k.

Come già anticipato, la selezione di un elemento di rango k può essere affrontata senza ordinare preventivamente l'intero input, sfruttando la capacità degli algoritmi di selezione di scartare progressivamente gli elementi che non possono corrispondere alla soluzione cercata.

L'obiettivo della tesi è quindi partire da queste osservazioni e concentrarsi sul caso in cui la quantità di memoria di lavoro disponibile sia molto limitata rispetto alla dimensione complessiva dell'input, studiando come sia possibile ricavarne la mediana attraverso l'approccio multi-pass proposto da Munro e Paterson.

1.2 Cenni storici

Il problema della selezione è stato oggetto di studio già negli anni Settanta. Un risultato fondamentale fu ottenuto nel 1973 da Manuel Blum, Robert Floyd, Vaughan Pratt, Ronald

Rivest e Robert Tarjan, che presentarono un algoritmo deterministico, denominato *PICK*, capace di selezionare un elemento di rango assegnato con un numero di confronti lineare rispetto alla dimensione dell'input anche nel caso peggiore [1]. Questo risultato ha posto le basi dell'approccio oggi comunemente presentato attraverso la tecnica della *median of medians*, che verrà approfondita nel capitolo successivo.

Nel 1980 J. Ian Munro e Michael S. Paterson affrontarono invece il problema della selezione considerando esplicitamente una quantità limitata di memoria interna e un input accessibile sequenzialmente. Nel loro articolo *Selection and Sorting With Limited Storage* si concentrarono in particolare sulla relazione tra memoria disponibile e numero di passate necessarie sull'input, proponendo un algoritmo di selezione basato sulla costruzione ricorsiva di campioni organizzati su più livelli [3]. Questo modello di riferimento e l'algoritmo multi-pass sviluppato dagli autori costituiscono il principale oggetto di studio di questa tesi.

1.3 Obiettivi e struttura della tesi

L'obiettivo principale dell'elaborato è studiare il problema del calcolo della mediana in condizioni di memoria limitata, concentrandosi sull'algoritmo multi-pass proposto da Munro e Paterson. Si parte dal problema classico della selezione e dall'algoritmo deterministico *median of medians*, utilizzato come riferimento per introdurre successivamente un modello in cui l'intero input non può essere mantenuto contemporaneamente in memoria e nasce quindi la necessità di adottare un metodo differente.

Dopo aver definito il modello computazionale, verrà prima fornita una visione generale dell'algoritmo di Munro e Paterson e successivamente verranno approfonditi la struttura di una singola passata, la costruzione dei campioni, la gestione del limite di memoria e la scelta dei filtri, o sentinelle, utilizzati per restringere progressivamente l'insieme dei candidati. L'algoritmo verrà infine descritto tramite pseudocodice, illustrandone il funzionamento, la correttezza e le condizioni di terminazione.

La seconda parte del lavoro è invece dedicata all'implementazione vera e propria dell'algoritmo in un linguaggio di programmazione e alla sua valutazione sperimentale. La sperimentazione avverrà attraverso test su dataset di diverse dimensioni, evidenziando in particolare il compromesso tra quantità di memoria disponibile e numero di passate richieste sull'input. Il comportamento osservato verrà quindi confrontato con i risultati teorici, così da ottenere un riscontro pratico sul trade-off che verrà analizzato nel corso dell'elaborato. Infine, viene presentato un possibile caso applicativo nel ‘‘miniworld’’ della Formula 1, con l'obiettivo di mostrare come il calcolo della mediana con memoria limita-

ta possa essere utilizzato in un contesto caratterizzato da acquisizione sequenziale dei dati e disponibilità di memoria ridotta.

Capitolo 2

Problema della mediana e algoritmo SELECT

2.1 Definizione del problema e notazione

In questa sezione vengono introdotte la notazione e le definizioni formali utilizzate nell'elaborato, mantenendo una convenzione coerente con quella adottata da Munro e Paterson, ripresa poi nella descrizione del loro algoritmo [3]. Sia:

$$A = (a_1, a_2, \ldots, a_N)$$

una sequenza di N elementi appartenenti a un dominio totalmente ordinato e si assume che questi siano tutti distinti:

$$a_i \neq a_j \qquad \forall\, i, j \in \{1, \ldots, N\},\ i \neq j$$

Non si fa alcuna ipotesi sull'ordine in cui gli elementi compaiono nell'input e se questi fossero già ordinati, il problema sarebbe banale. Si indichi con:

$$a_{(1)} > a_{(2)} > \ldots > a_{(N)}$$

la stessa collezione di elementi disposti in ordine decrescente.

Osservazione 2.1. Le parentesi nell'indice distinguono la posizione di un elemento nell'ordinamento da quella che esso occupa nella sequenza in ingresso: $a_{(i)}$ è l'elemento che occuperebbe la posizione i se A fosse ordinata, mentre a_i è l'i-esimo elemento della sequenza iniziale.

Definizione 2.1 (Rango)**.** Il *rango* di un generico elemento $a_{(i)}$ è la posizione i che esso occupa nella sequenza ordinata in modo decrescente. Ad esempio, $a_{(1)}$ è l'elemento più grande, $a_{(2)}$ il secondo più grande e, in generale, $a_{(k)}$ è il k-esimo elemento più grande.

Definizione 2.2 (Problema della selezione)**.** Dato un intero k con $1 \leq k \leq N$, il *problema della selezione* consiste nel determinare l'elemento $a_{(k)}$ senza ordinare l'intera sequenza A.

Definizione 2.3 (Mediana)**.** Si dice *mediana* di A l'elemento $a_{(k)}$ ottenuto risolvendo il problema della selezione per:

$$k = \left\lceil \frac{N}{2} \right\rceil$$

ossia l'elemento con indice «centrale» se N è dispari e il maggiore dei due «centrali» se N è pari.

Senza perdere di generalità si adotta l'ordinamento decrescente, e ne deriva la convenzione del k-esimo elemento più grande, per essere allineati con la formulazione di Munro e Paterson.

2.2 Selezione mediante pivot

Un elemento fondamentale degli algoritmi di selezione è il *pivot*, ovvero un elemento della sequenza scelto come riferimento per suddividere l'input. Una volta fissato un pivot x, ogni altro elemento viene confrontato con esso: poiché dalle ipotesi gli elementi sono distinti, ciascun elemento viene classificato come maggiore o minore di x. Il pivot occupa dunque una posizione univocamente determinata all'interno della sequenza ordinata.

Questo meccanismo permette di stabilire in quale delle due partizioni si trovi l'elemento di rango k cercato, eliminando completamente l'altra dalla ricerca, in modo da «sfoltire» la dimensione del problema e ricondurlo a un problema dello stesso tipo definito su un numero inferiore di elementi.

La scelta del pivot ha un ruolo determinante nell'efficienza dell'algoritmo ed esistono varie tipologie di filosofie per la scelta di esso. Una possibile strategia consiste nel selezionarlo casualmente, come avviene nelle versioni randomizzate di QuickSort e QuickSelect . Tale approccio talvolta è utile perché rende poco probabile ottenere ripetutamente partizioni fortemente sbilanciate e porta in genere a buone prestazioni nella pratica. La scelta casuale, tuttavia, non elimina il caso peggiore, ma cerca solo di evitarlo: se per esempio il pivot selezionato si trova ripetutamente vicino a uno degli estremi dell'ordinamento, a ogni iterazione viene eliminato soltanto un sottoinsieme molto ridotto di elementi, e la complessità può diventare quadratica rispetto alla taglia dell'input [2].

Nel problema di interesse però si vuole ottenere una garanzia deterministica anche sul caso peggiore. È quindi necessario scegliere il pivot mediante una procedura che garantisca l'eliminazione di una frazione sufficientemente grande degli elementi a ogni passo, in modo indipendente dalla loro disposizione iniziale. L'algoritmo *SELECT* utilizza per questo scopo la tecnica della *median of medians*: nella quale il pivot non è quindi scelto arbitrariamente o casualmente, ma viene costruito in modo da non essere troppo vicino agli estremi della sequenza ordinata (la costruzione del pivot e le garanzie fornite da questa scelta verranno approfondite nella sezione successiva).

Una volta scelto il pivot x, è possibile determinarne il rango contando quanti elementi della sequenza risultano maggiori di esso. Per farlo, si separano gli elementi dell'input in due insiemi, distinguendo quelli maggiori del pivot da quelli minori, denotando:

$$G_x = \{a_i \in A \mid a_i > x\}$$

l'insieme degli elementi maggiori del pivot:

$$L_x = \{a_i \in A \mid a_i < x\}$$

l'insieme degli elementi minori del pivot.

Osservazione 2.2. Poiché per ipotesi tutti gli elementi della sequenza sono distinti, nessun elemento può essere contemporaneamente maggiore e minore del pivot. Dunque i due insiemi sono quindi disgiunti e vale:

$$G_x \cap L_x = \varnothing$$

Posto $m = |G_x|$, nell'ordinamento decrescente il pivot occupa necessariamente la posizione $m + 1$. Confrontando k con m si distinguono tre casi, mutuamente esclusivi ed esaustivi:

- se $k = m + 1$, il pivot coincide con l'elemento cercato e la selezione termina;
- se $k \leq m$, il k-esimo elemento più grande appartiene necessariamente a G_x: la ricerca prosegue esclusivamente su tale partizione, mantenendo invariato il rango k;
- se $k > m + 1$, l'elemento cercato appartiene necessariamente a L_x. Poiché vengono esclusi dalla ricerca i m elementi di G_x e il pivot stesso, il rango da cercare nella nuova partizione verrà «shiftato» di un certo offset:

$$k' = k - (m + 1)$$

La selezione poi verrà applicata ricorsivamente alla sola partizione che contiene l'elemento desiderato, scartando definitivamente tutti gli altri elementi. Si può osservare che la qualità della suddivisione ottenuta, e quindi il numero di elementi eliminabili a ogni passo, dipende strettamente dalla scelta del pivot.

2.3 Algoritmo SELECT: median of medians

L'algoritmo *SELECT* risolve il problema della selezione costruendo deterministicamente il pivot tramite la tecnica della *median of medians*. L'idea alla base è quella di non scegliere direttamente un elemento dell'input come pivot, ma di ricavarlo attraverso una serie di mediane intermedie, in modo da ottenere una suddivisione sufficientemente bilanciata della sequenza [1, 2].

Quindi data una sequenza di N elementi, per prima cosa la si suddivide in sottogruppi di cinque elementi ciascuno, con al più un ultimo gruppo contenente meno di cinque elementi (senza padding). Si ottengono così:

$$\left\lceil \frac{N}{5} \right\rceil \quad \text{gruppi} \quad G_1, G_2, \ldots, G_{\lceil N/5 \rceil}$$

Gli elementi di ciascun gruppo G_i, con $1 \leq i \leq \lceil \frac{N}{5} \rceil$, vengono ordinati e ne viene individuata la mediana $m_i = \text{median}(G_i)$. Si ottiene in questo modo una nuova sequenza formata dalle mediane dei singoli sottogruppi:

$$M = (m_1, m_2, \ldots, m_{\lceil N/5 \rceil})$$

Su questa nuova sequenza viene poi applicato ricorsivamente lo stesso algoritmo *SELECT*, con l'obiettivo di determinarne la mediana, ossia:

$$x = \textsc{Select}\left(M, \left\lceil \frac{|M|}{2} \right\rceil\right)$$

L'elemento x così ottenuto prende il nome di *mediana delle mediane* e viene promosso come *pivot* per la partizione dell'input originale.

Fissato x, la sequenza originale viene suddivisa negli insiemi G_x e L_x, come definiti precedentemente. A questo punto, confrontando il rango cercato k con il rango occupato dal pivot, è possibile stabilire se x coincide con il goal oppure su quale delle due partizioni proseguire ricorsivamente la ricerca, applicando gli stessi tre casi visti in precedenza. L'altra partizione può essere scartata definitivamente.

La particolarità dell'algoritmo sta quindi nel modo in cui viene scelto il pivot. Per il modo in cui viene scelto x, non può trovarsi arbitrariamente vicino agli estremi dell'ordinamento. La struttura dei sottogruppi da cinque assicura infatti che un numero sufficientemente grande di elementi sia maggiore di x e, allo stesso tempo, che un numero sufficientemente grande sia minore di x. Questa proprietà impedisce che la partizione ricorsiva mantenga quasi tutti gli elementi dell'input e permette di ottenere una garanzia sul tempo di esecuzione anche nel caso peggiore.

La garanzia quantitativa fornita dalla scelta della mediana delle mediane verrà presentata nella prossima sezione, insieme alla complessità temporale dell'algoritmo.

2.4 Pseudocodice e complessità

Le operazioni descritte nelle sezioni precedenti possono essere formalizzate nel seguente pseudocodice:

Algorithm 1 SELECT

```
 1: function SELECT(A, k)
 2:     N ← |A|
[B]
 3:     if N ≤ 5 then
 4:         *ordina A in ordine decrescente*
 5:         return A[k]
 6:     *suddividi A in ⌈N/5⌉ gruppi G_1, ..., G_⌈N/5⌉*
 7:     for i ← 1 to ⌈N/5⌉ do
 8:         *ordina G_i in ordine decrescente*
 9:         m_i ← G_i [⌈|G_i|/2⌉]
10:     M ← (m_1, m_2, ..., m_⌈N/5⌉)
[R]
11:     x ← SELECT(M, ⌈|M|/2⌉)
12:     G_x ← {a ∈ A | a > x}
13:     L_x ← {a ∈ A | a < x}
14:     m ← |G_x|
15:     if k = m + 1 then
16:         return x
17:     else if k ≤ m then
[R]
18:         return SELECT(G_x, k)
19:     else
[R]
20:         return SELECT(L_x, k − (m + 1))
```

Rispettivamente le etichette indicano: **[B]** il caso base **[R]** la chiamata ricorsiva all'algoritmo SELECT.

L'analisi del tempo di esecuzione di *SELECT* nel caso peggiore può essere espressa mediante la seguente ricorrenza:

$$T(N) \leq \begin{cases} \Theta(1) & N \leq 80 \\ T\left(\lceil \frac{N}{5} \rceil\right) + T\left(\frac{7N}{10} + 6\right) + O(N) & N > 80 \end{cases}$$

Dove il primo termine è dovuto alla selezione della mediana all'interno della sequenza M, mentre il secondo rappresenta la dimensione massima della partizione sulla quale procede la ricerca, nel caso peggiore. Infine il termine $O(N)$ è un upper bound delle operazioni necessarie alla costruzione dei sottogruppi, all'individuazione delle relative mediane e al partizionamento rispetto al pivot. Risolvendo la ricorrenza con la tecnica dell' *unfolding*, cioè ripetendone più volte la definizione implicita, si ottiene:

$$T(N) = O(N)$$

Quindi l'algoritmo *SELECT* permette di risolvere deterministicamente il problema della selezione con un tempo lineare anche nel caso peggiore [1, 2].

2.5 Limiti del modello classico

L'algoritmo *SELECT* mostra che il problema della selezione può essere risolto deterministicamente in tempo lineare. Tale risultato presuppone tuttavia, implicitamente, un modello di accesso ai dati coerente con la macchina RAM: l'intero input deve risiedere in memoria primaria, in modo da poter accedere liberamente ai suoi elementi, partizionarlo ripetutamente e mantenere le sottosequenze coinvolte nelle successive chiamate ricorsive.

Si osservi che questa assunzione cessa di essere vera quando la dimensione dell'input è molto maggiore della memoria di lavoro disponibile. In tal caso non è più possibile mantenere contemporaneamente in memoria tutti gli elementi dell'input, né applicare direttamente le operazioni di partizionamento su cui si basa *SELECT*. Il solo tempo di esecuzione non è più un modello di costo coerente del problema, di conseguenza diventa necessario tenere conto anche della quantità di memoria disponibile (nel dispositivo in cui viene eseguito il codice) e delle modalità con cui è consentito accedere ai dati.

Lo studio di Munro e Paterson è incentrato proprio su questo scenario, considerando un input accessibile esclusivamente in modo sequenziale, come un nastro a lettura unidirezionale, e una memoria interna di dimensione ridotta rispetto all'input. Sotto queste

ipotesi dunque, il numero di passate sull'input diventa una risorsa computazionale al pari della memoria di lavoro e rileggere più volte la sequenza permette di ricavare, passata dopo passata, l'informazione che nel modello classico si otterrebbe con un unico accesso casuale ai dati, al costo però di un tempo di esecuzione complessivo maggiore [3]. Il prossimo capitolo introdurrà formalmente questo modello computazionale e mostrerà come il problema della selezione possa essere riformulato in presenza di tali vincoli.

Capitolo 3

Selezione con memoria limitata

3.1 Modello computazionale e notazione

In questa sezione si introducono le nuove grandezze necessarie per estendere il modello definito nei capitoli precedenti e descrivere più nel dettaglio l'algoritmo di selezione con memoria limitata. Si parte da:

Definizione 3.1 (Nastro di input)**.** La sequenza A è memorizzata su un dispositivo di lettura, detto *nastro*, che consente di leggere gli elementi $a_1, a_2, \ldots, a_N$ solamente in modo sequenziale e unidirezionale, nell'ordine in cui sono memorizzati. Gli elementi del nastro non possono essere modificati e, una volta superata una determinata posizione, non è possibile tornarci senza prima riportare il nastro alla posizione iniziale.

Definizione 3.2 (Memoria disponibile)**.** Sia $S \in \mathbb{N}$ il numero di celle di memoria ad accesso casuale a disposizione dell'algoritmo. Un elemento letto dal nastro può essere trasferito in una delle celle disponibili e due elementi possono essere confrontati solo se entrambi si trovano, nello stesso momento, in memoria.

Definizione 3.3 (Passata)**.** Si dice *passata* una scansione completa del nastro, dalla posizione iniziale a_1 alla posizione finale a_N. Al termine di una passata il nastro viene riportato alla posizione iniziale, senza eseguire nel mentre ulteriori letture, in modo da poter iniziare con un'eventuale nuova passata.

Definizione 3.4 (Numero di passate)**.** Sia $P \in \mathbb{N}$ il numero complessivo di passate eseguite sul nastro prima che l'algoritmo restituisca l'elemento di rango k.

Per restringere via via la partizione dell'ordinamento in cui può trovarsi l'elemento cercato, l'algoritmo mantiene in memoria una coppia di elementi, chiamati *filtri*.

Definizione 3.5 (Filtri)**.** Si dicono *filtri* due valori $f_{\min}$ e $f_{\max}$ mantenuti in memoria durante l'esecuzione, tali che:

$$f_{\min} \leq a_{(k)} \leq f_{\max}$$

I filtri delimitano quindi un intervallo dell'ordinamento che contiene sempre l'elemento cercato, fungendo da sentinelle. All'inizio dell'algoritmo, non essendo ancora disponibile alcuna informazione sulla posizione di $a_{(k)}$, vengono inizializzati di default a $-\infty$ e $+\infty$.

Definizione 3.6 (Candidati)**.** Fissata una coppia di filtri $f_{\min}$ e $f_{\max}$, si può definire l'insieme dei *candidati* della passata come:

$$C = \{a_i \in A \mid f_{\min} < a_i < f_{\max}\}$$

e si indica con $n = |C|$, la cardinalità dell'insieme, quindi il numero di elementi strettamente compresi tra i due filtri.

Osservazione 3.1. Gli elementi di C sono quelli che, allo stato attuale, non possono ancora essere esclusi dalla ricerca. I due filtri non appartengono a C e vengono mantenuti separatamente in memoria, dato che uno di essi potrebbe già coincidere con $a_{(k)}$.

Si noti inoltre che il pedice degli elementi candidati non è racchiuso tra parentesi, quindi gli elementi di C indicati come a_i, mantengono ancora la posizione che occupavano nella sequenza sul nastro.

Il problema della selezione, dunque si riduce a determinare $a_{(k)}$ usando al più S celle di memoria e P passate sul nastro. Inoltre si può formulare il seguente invariante per l'algoritmo.

Invariante 3.1 (Invariante per i filtri)**.** Siano $f_{\min}^{(p)}$ e $f_{\max}^{(p)}$ i filtri utilizzati all'inizio di una generica passata p. Per ogni passata eseguita dall'algoritmo deve valere:

$$f_{\min}^{(p)} \leq a_{(k)} \leq f_{\max}^{(p)}$$

di conseguenza, l'elemento di rango k non viene mai escluso dall'intervallo di ricerca durante l'esecuzione dell'algoritmo.

Raccolte queste definizioni, lo scenario in cui opera l'algoritmo può essere descritto nel suo insieme dalla Figura 3.1. La sequenza A non risiede in memoria, ma su un nastro che il programma legge un elemento alla volta, solo in avanti. Durante la lettura il programma può copiare alcuni elementi nelle S celle di memoria e, al termine della computazione, restituisce l'elemento di rango k. Se l'informazione raccolta non è sufficiente, il nastro viene riavvolto e si esegue una nuova passata.

Questo scenario corrisponde a quello che oggi viene comunemente chiamato *modello di streaming* nella sua variante *a più passate*, in cui i dati sono visti come un flusso che può essere letto soltanto in sequenza, eventualmente più volte, mentre la memoria di lavoro è molto più piccola dell'input. La differenza rispetto al modello RAM, considerato nel Capitolo 2 per SELECT, è evidente. Nel modello RAM l'intero input risiede in memoria e si può accedere direttamente a qualsiasi elemento, per cui la misura di costo è il tempo di esecuzione. Nel modello di streaming, invece, l'accesso ai dati è vincolato all'ordine del nastro e lo spazio è limitato, per cui le risorse da misurare diventano la memoria S e il numero di passate P.

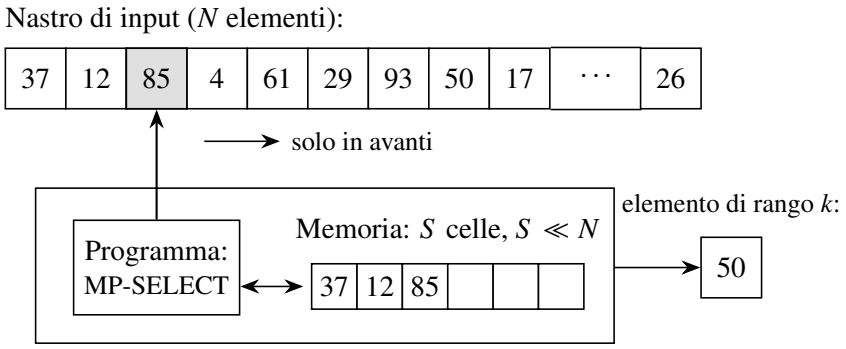


Figura 3.1: Modello di calcolo considerato. Nella figura è evidenziato l'elemento in lettura, appena copiato in memoria.

Osservazione 3.2 (Analogia con la Macchina di Turing)**.** Il modello può essere interpretato anche come *k-tape Turing Machine*, cioè una Macchina di Turing a k nastri con $k = 1$. È dunque dotata di un nastro di input idealmente illimitato, di un nastro di output con un solo elemento $a_{(k)}$, e di un unico nastro di lavoro, che corrisponde alla memoria di S celle. Rispetto alla macchina di Turing classica vi sono però due differenze. La prima è che la testina del nastro di input può muoversi solo in avanti, e tornare all'inizio del nastro equivale a iniziare una nuova passata. La seconda è che ogni cella del nastro di lavoro contiene un elemento completo della sequenza, e non un singolo simbolo, ed è accessibile direttamente.

3.2 Struttura generale dell'algoritmo

L'algoritmo di Munro e Paterson sostituisce il partizionamento diretto di SELECT con una riduzione progressiva dell'intervallo individuato dai filtri. L'algoritmo che segue è quello proposto dagli autori per il caso in cui la memoria sia sufficientemente grande, precisamente:

$$S \in \Omega((\log N)^2)$$

All'inizio, i filtri sono inizializzati di default a $-\infty$ e $+\infty$ e quindi $n = N$ e tutti gli elementi dell'input sono considerati come candidati. L'algoritmo esegue in seguito una successione di passate complete sul nastro. Durante ciascuna passata avvengono due cose contemporaneamente: si raccolgono le informazioni necessarie a determinare il rango esatto dei filtri correnti, e con gli elementi rimasti candidati si costruisce un campione che può «fittare» la memoria disponibile.

Il campione non contiene tutti i candidati, ma solo un sottoinsieme, scelto in modo da rimanere comunque rappresentativo della distribuzione originale. Da questo campione, poi, si ricavano due nuovi filtri, più vicini tra loro rispetto ai precedenti ma ancora tali da contenere $a_{(k)}$. A questo punto il nastro viene riavvolto e si procede con una nuova passata usando i filtri appena calcolati, n diminuisce così di passata in passata, senza che in nessun istante sia mai necessario tenere in memoria l'intera sequenza di candidati.

La Figura 3.2 mostra questo processo su un piccolo esempio. Gli elementi della sequenza sono disposti in ordine crescente e ogni riga rappresenta lo stato dell'algoritmo dopo p passate. Si indica con n_p il numero di candidati compresi tra i filtri ottenuti al termine della passata p, inizializzando $n_0 = N$. Per $p = 0$ i filtri valgono $-\infty$ e $+\infty$ e tutti gli elementi sono candidati. Dopo la prima passata i filtri si avvicinano e gli elementi che restano fuori dall'intervallo, indicati in grigio, vengono scartati definitivamente. Dopo la seconda passata l'intervallo si restringe ulteriormente.

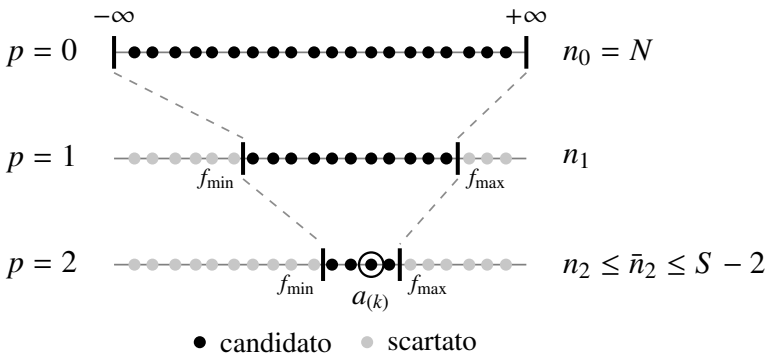


Figura 3.2: L'elemento cercato $a_{(k)}$ resta sempre tra i filtri, mentre il numero di candidati n_p diminuisce fino a scendere sotto la soglia $S - 2$.

L'algoritmo interrompe il campionamento quando il limite superiore sul numero di candidati scende a $S - 2$ o meno, a questo punto ci si ritrova nel caso base e gli elementi rimasti possono essere letti e mantenuti tutti contemporaneamente in memoria (con i due filtri), e la selezione si conclude con un'ultima passata diretta sui pochi elementi restanti. Nota che è anche possibile che l'esecuzione termini prima, ad esempio se durante il calcolo del rango di un filtro ci si accorge che questo coincide proprio con $a_{(k)}$.

I tre ingredienti da definire sono dunque: come si costruisce il campione di una passata, verificare che non si eccedano mai le S celle disponibili, e come da tale campione si ricava la nuova coppia di filtri. Questi passaggi verranno descritti e implementati nel dettaglio nelle prossime sezioni.

3.3 Struttura di una passata

Si fissi una passata che inizia con filtri $f_{\min}$, $f_{\max}$ e con n candidati. Il nastro va comunque letto per intero, perché non c'è modo di accedere solo alle posizioni che contengono elementi di C: ogni elemento letto viene confrontato con i filtri correnti.

Gli elementi che cadono fuori dall'intervallo $(f_{\min}, f_{\max})$ non entrano nel campione, ma sono necessari ad aggiornare i conteggi con cui si calcola il rango dei filtri. Gli elementi che, invece, cadono dentro l'intervallo, cioè i candidati, costituiscono invece la popolazione da cui si costruisce il campione della passata. Terminata la scansione, il campione ottenuto viene utilizzato per scegliere i filtri della passata successiva.

3.3.1 Costruzione del campione

Il campione deve conciliare due vincoli che vanno in direzioni opposte: occupare uno spazio molto più piccolo rispetto agli n candidati di partenza, ma conservare comunque abbastanza informazione sulla loro distribuzione nell'ordinamento da poter restringere l'intervallo di ricerca in modo sicuro. Munro e Paterson risolvono questo compromesso con una costruzione ricorsiva, organizzata su più livelli [3], definita come segue:

Definizione 3.7 (Campione di livello i)**.** Sia s un intero positivo e pari. Un *campione di livello i* è una sequenza ordinata di s elementi, ottenuta da una popolazione di $2^i s$ elementi candidati secondo la costruzione ricorsiva:

- **Caso base ($i = 0$)**

  Quando $i = 0$ la popolazione si riduce a $2^0 s = s$ elementi, il campione di livello 0 coincide allora con l'intera popolazione, semplicemente ordinata in modo decrescente. In questo livello non vi è ancora nessuna perdita di informazione, perché non è stato scartato nulla.

- **Passo ricorsivo ($i + 1$)**

  Per la costruzione di un campione di livello $i + 1$ si parte invece da una popolazione più ampia, di $2^{i+1} s$ elementi, che viene divisa in due metà consecutive rispetti-

vamente da $2^i s$ elementi ciascuna, seguendo l'ordine con cui i candidati vengono incontrati sul nastro.

Da ogni metà si ricava, ricorsivamente, un campione di livello i, ottenendo così due campioni ordinati di s elementi ciascuno. Questi due campioni non possono però essere semplicemente uniti, perché si otterrebbe una sequenza di $2s$ elementi, mentre un campione deve mantenerne sempre s. Prima di combinarli è quindi necessario ridurli tramite l'operazione di *thinning*.

Definizione 3.8 (Thinning)**.** Sia:

$$X = (x_1, x_2, \ldots, x_s)$$

un campione di s elementi, con s pari, ordinato in modo decrescente. L'operazione di *thinning* consiste nel tenere un elemento ogni due, dunque mantenendo solo quelli che occupano una posizione pari:

$$\text{thin}(X) = (x_2, x_4, x_6, \ldots, x_s)$$

La dimensione del campione viene quindi ridotta da s a $s/2$ elementi.

Applicato ai due campioni di livello i, il thinning li riduce a $s/2$ elementi ciascuno e a questo punto possono essere ricombinati con un *merge*, tornando ad avere una sequenza ordinata di s elementi complessivi. Il risultato totale è il campione di livello $i + 1$ cercato.

L'intero procedimento è di facile comprensione se visualizzato come un albero binario (dal basso verso l'alto), mostrato in Figura 3.3, dove le foglie corrispondono ai campioni di livello 0, ciascun nodo interno applica il thinning ai due sottocampioni dei figli e li fonde con un merge, e la radice dell'albero, raggiunta al livello più alto r è il campione finale usato per aggiornare i filtri.

Osservazione 3.3. Pur essendo organizzata su r livelli, la costruzione del campione non richiede una passata per livello: tutti i livelli da 0 a r vengono costruiti durante un'unica scansione del nastro. L'albero della Figura 3.3 rappresenta infatti soltanto le dipendenze *logiche* tra i sottocampioni, che non devono essere conservati tutti contemporaneamente in memoria.

Per capire come varia l'occupazione della memoria durante la ricorsione, si può seguire la costruzione di due sottocampioni consecutivi. Al livello 0 un buffer di lavoro si riempie progressivamente, un candidato alla volta, fino a contenere s elementi. Il campione così ottenuto viene ordinato e, quando deve essere combinato con un altro campione, ridotto tramite *thinning* a $s/2$ elementi. Questa prima metà viene conservata temporaneamente,

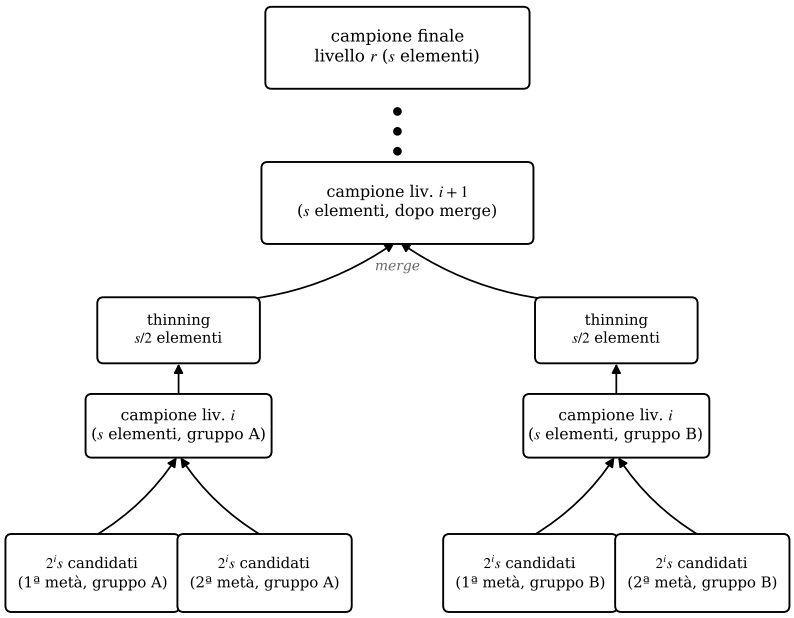


Figura 3.3: Costruzione ricorsiva del campione: a ogni livello i due sottocampioni vengono «assottigliati» con il thinning e poi riuniti con un merge, fino ad arrivare al campione finale di livello r.

mentre lo stesso buffer di lavoro viene riutilizzato per costruire la seconda. Una volta disponibile anche la seconda metà, il *merge* produce un nuovo campione di s elementi, riutilizzando lo stesso spazio di lavoro. A quel punto la prima metà in attesa non serve più e le relative celle tornano disponibili.

Il procedimento si ripete anche ai livelli superiori: ciascun livello intermedio può quindi avere uno spazio di attesa vuoto oppure occupato da un solo sottocampione già assottigliato di $s/2$ elementi. Non si accumulano due campioni completi di s elementi per livello, perché il primo viene ridotto prima di costruire il secondo. In questo modo, oltre al buffer condiviso di s elementi e alle due celle dei filtri, servono al più $s/2$ celle per ciascuno degli r livelli intermedi [3, 4].

Resta da stabilire a quale livello r convenga fermare questa costruzione. All'inizio della passata non si conosce ancora il numero esatto di candidati, ma solo un limite superiore $\bar{n}$ su di esso, si sceglie dunque il più piccolo r la cui popolazione, di $2^r s$ elementi, sia in grado di contenere anche nel caso peggiore tutti i candidati previsti. Deve quindi valere:

$$\bar{n} \leq 2^r s$$

da cui, risolvendo per r:

$$r = \left\lceil \log_2 \left(\frac{\bar{n}}{s} \right) \right\rceil$$

Il campione di livello r così ottenuto è il campione finale restituito dalla passata (la radice dell'albero raffigurato).

Se il numero di candidati effettivi è strettamente minore di $2^r s$ (il che accade la maggior parte delle volte, a meno che $\bar{n}$ non sia esattamente una potenza di due volte s), la popolazione viene completata con degli elementi fittizi (con un'operazione di *padding*), allo scopo di raggiungere la dimensione richiesta dalla struttura ricorsiva.

3.3.2 Verifica dei vincoli della memoria

Naturalmente, questa costruzione è valida solo se riesce a stare entro le S celle di memoria effettivamente disponibili, e conviene verificarlo prima di proseguire. Come descritto nella sezione precedente, la costruzione ricorsiva riutilizza un unico buffer di lavoro di s elementi e conserva, per ciascuno degli r livelli intermedi, al più un sottocampione già assottigliato di $s/2$ elementi in attesa di essere combinato con il successivo.

Lo spazio complessivo richiesto è dunque: quello di un sottocampione per ciascuno degli r livelli, più un campione di lavoro di s elementi, più le due celle occupate dai filtri, in totale:

$$\frac{rs}{2} + s + 2 \leq S$$

Questo vincolo lega s e r tra loro, che non possono quindi essere scelti in modo indipendente, ad esempio un campione più grande occupa più spazio, ma permette anche di fermarsi a un livello r più basso. Munro e Paterson risolvono il compromesso fissando:

$$s = 2 \left\lfloor \frac{S}{2 \log_2 \bar{n}} \right\rfloor \qquad r = \left\lceil \log_2 \left(\frac{\bar{n}}{s} \right) \right\rceil$$

una scelta che mantiene s pari e rispetta il vincolo per S sufficientemente grande [3]. L'andamento risultante è abbastanza intuitivo, se s cresce proporzionalmente a $S/\log \bar{n}$ (più memoria è disponibile) più grande può essere il campione; con poca memoria, al contrario, il campione resta necessariamente piccolo e servono più livelli, cioè un r maggiore.

3.3.3 Scelta della nuova coppia di filtri

A questo punto il campione è pronto, ma resta da capire come usarlo per restringere l'intervallo di ricerca senza rischiare, nel farlo, di perdere $a_{(k)}$. Il procedimento si poggia sull'idea che ogni elemento del campione occupa all'interno della popolazione da cui è

stato estratto, una posizione che non è nota con esattezza, ma che è comunque racchiusa in un intervallo di cui si conoscono gli estremi. Prima di formalizzarla, però, conviene fissare le quantità coinvolte.

Definizione 3.9 (Limiti di rango del campione)**.** Per il j-esimo elemento più grande di un campione di livello i, si indicano con L_{ij} e M_{ij} rispettivamente il numero minimo e il numero massimo di elementi della popolazione di partenza che possono trovarsi, nell'ordinamento, sopra di esso. Munro e Paterson dimostrano che tali bound si calcolano esplicitamente come:

$$L_{ij} = j2^i - 1, \qquad M_{ij} = (i + j - 1)2^i$$

In altre parole, non si sa esattamente quanti elementi della popolazione precedano un dato elemento del campione, ma si sa che non sono più di M_{ij} e non meno di L_{ij} [3]. È questa «doppia garanzia» che consente di usare il campione per restringere l'intervallo di ricerca senza commettere errori.

La parte (a) della Figura 3.4 dà una rappresentazione visiva della garanzia sulla «retta del rango», poiché L_{ij} e M_{ij} contano gli elementi che precedono x_j, il rango effettivo di x_j all'interno della popolazione è compreso tra $L_{ij}+1$ e $M_{ij}+1$. L'elemento x_j non ha quindi una posizione nota, ma è comunque vincolato a non poter cadere fuori da tale intervallo. Per confrontare L_{ij} e M_{ij} con il rango k dell'elemento cercato, c'è però un ostacolo da chiarire: k è definito rispetto all'intera sequenza A, mentre L_{ij} e M_{ij} descrivono posizioni *all'interno della popolazione corrente* dei soli candidati. Le due grandezze sono riferite quindi a scale diverse, e di conseguenza non sono ancora direttamente confrontabili.

Per convertirle sulla stessa scala serve il rango dei filtri, che viene ricavato proprio durante la scansione del nastro.

Definizione 3.10 (Rango di un filtro)**.** Per un filtro reale x, ricordando che gli elementi di A sono tutti distinti, si definisce il suo rango come:

$$\rho(x) = 1 + |\{a_i \in A \mid a_i > x\}|$$

Per il filtro ideale (iniziale) si pone per convenzione $\rho(+\infty) = 0$, coerentemente con il fatto che nessun elemento della sequenza può superarlo.

Il valore $\rho(f_{\max})$ si ottiene semplicemente contando, durante la lettura del nastro, quanti elementi sono maggiori di $f_{\max}$. Poiché tutti i candidati occupano, per costruzione, le posizioni immediatamente successive a $f_{\max}$ nell'ordinamento di A, sottraendo $\rho(f_{\max})$ al rango k si ottiene esattamente il rango che $a_{(k)}$ avrebbe se si contassero solo i candidati della popolazione corrente, a partire da $f_{\max}$.

Definizione 3.11 (Rango relativo)**.** Si dice *rango relativo* di $a_{(k)}$ rispetto alla popolazione corrente dei candidati la quantità:

$$k_C = k - \rho(f_{\max})$$

Il passaggio da k a k_C non introduce quindi nessuna incognita nuova, k_C resta il rango dell'elemento cercato, soltanto misurato a partire dall'inizio della popolazione corrente invece che dall'inizio di tutta la sequenza. È proprio questa la convenzione comune che permette di confrontare k_C con i limiti L_{ij} e M_{ij} della Definizione 3.9.

Fatta questa conversione, i nuovi filtri possono essere scelti direttamente a partire dai limiti della Definizione 3.9. Indicando con $x_1 > x_2 > \ldots > x_s$ il campione finale di livello r, per il filtro superiore si cerca il più grande indice u tale che, anche nel caso peggiore, sopra x_u non possano trovarsi più di $k_C - 1$ elementi. Si cerca quindi:

$$M_{ru} = (r + u - 1)2^r \leq k_C - 1$$

da cui, risolvendo per u:

$$u = \left\lceil \frac{k_C}{2^r} \right\rceil - r$$

Analogamente per il filtro inferiore si cerca invece il più piccolo indice v tale che sopra x_v vi siano sicuramente almeno $k_C - 1$ elementi. Deve quindi valere:

$$L_{rv} = v2^r - 1 \geq k_C - 1$$

da cui, risolvendo per v:

$$v = \left\lceil \frac{k_C}{2^r} \right\rceil$$

Infine si pone $f'_{\max} = x_u$ e $f'_{\min} = x_v$ e per costruzione, $a_{(k)}$ deve trovarsi tra i due nuovi filtri oppure coincidere con uno di essi. L'invariante 3.1 continua quindi a valere dopo l'aggiornamento, grazie alla conversione effettuata precedentemente.

La parte (b) della Figura 3.4 mostra questa conclusione sulla stessa «retta del rango» usata sopra per (a), dove gli indici u e v individuano gli elementi x_u e x_v del campione finale, che diventano i nuovi filtri $f'_{\max}$ e $f'_{\min}$. La fascia evidenziata in nero tra x_u e x_v è l'intervallo entro cui $a_{(k)}$ è garantito, in base a come u e v sono stati scelti a partire dalla Definizione 3.9.

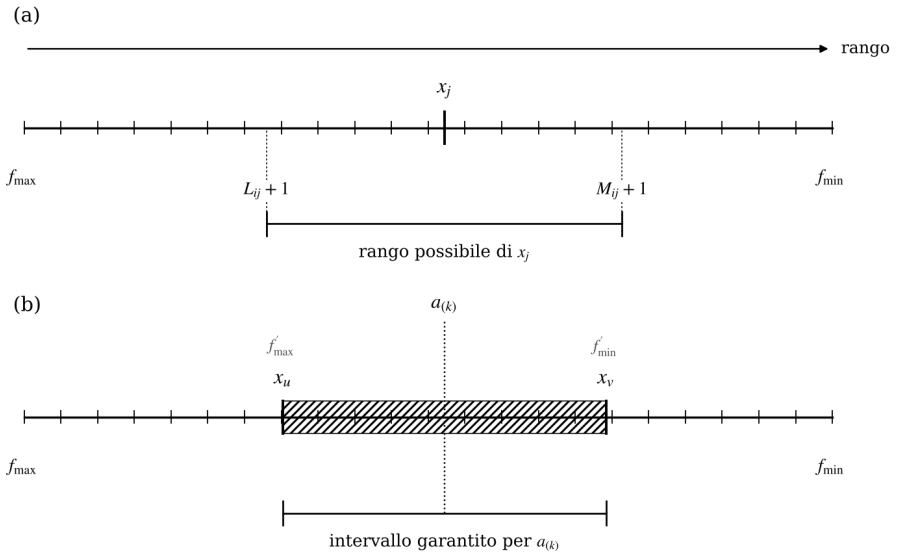


Figura 3.4: Interpretazione grafica delle quantità introdotte in questa sezione. Nel pannello (a), $L_{ij} + 1$ e $M_{ij} + 1$ sono gli estremi dell'intervallo in cui può trovarsi il rango di un generico elemento x_j del campione. Nel pannello (b), gli indici u e v individuano gli elementi x_u e x_v del campione finale di livello r che diventano i nuovi filtri $f'_{\max}$ e $f'_{\min}$.

Resta solo da definire quale sia stato effettivamente il guadagno ottenuto con la passata appena conclusa, cioè quanti candidati possano ancora trovarsi tra i nuovi filtri x_u e x_v. Applicando ancora la Definizione 3.9, ma agli indici u e v, il numero massimo di elementi compresi tra i due risulta essere:

$$M_{rv} - L_{ru} - 1 = (2r - 1)2^r$$

sostituendo con i valori di r e s fissati nella Sezione 3.3.2, si trova:

$$(2r - 1)2^r \in O\left(\frac{\bar{n}(\log \bar{n})^2}{S}\right)$$

Una singola passata riduce dunque l'upper bound sul numero di candidati da $\bar{n}$ a $O(\bar{n}(\log \bar{n})^2/S)$, ed è proprio questa riduzione, ottenuta passata dopo passata ripetendo lo stesso procedimento in modo deterministico, a far convergere l'algoritmo verso una popolazione abbastanza piccola da poter essere contenuta tutta in memoria, in un'unica ultima passata.

3.4 Pseudocodice dell'algoritmo

Le idee esposte nelle sezioni precedenti sono la teoria necessaria per formalizzare uno pseudocodice per questo algoritmo.

Prima di scrivere il codice, però, conviene chiarire un punto lasciato finora sullo sfondo e anticipato nella Sezione 3.3.1: all'inizio di una passata non si conosce ancora il numero esatto n di candidati, perché per conoscere quanti elementi ricadano tra i filtri sarebbe necessario esaminare il nastro. Per fissare i parametri s e r della costruzione del campione basta quindi conoscere un limite superiore su tale numero, indicato con $\bar{n}$. Nella prima passata questo limite è semplicemente $\bar{n} = N$, invece nelle passate successive il bound ricavato nella Sezione 3.3.3 fornisce direttamente il valore da usare per quella successiva.

Osservazione 3.4. L'introduzione di $\bar{n}$ non richiede quindi una passata aggiuntiva dedicata al solo conteggio dei candidati, ma si tratta di un limite superiore già noto prima che la scansione abbia inizio. Mentre $n = |C|$ resta, come prima, il numero effettivo di candidati osservato durante la passata.

3.4.1 Realizzazione del campione

Lo pseudocodice della costruzione del campione deriva dalla Definizione 3.7 ed è, in sostanza, una traduzione diretta della costruzione ricorsiva descritta nella Sezione 3.3.1. Per rendere più «operativa» la lettura si introduce la primitiva NextCand, che prosegue la scansione dal punto corrente fino a incontrare il candidato successivo. È PASS, non SAMPLE, a posizionare il nastro all'inizio della sequenza e a inizializzare NextCand, una sola volta, all'avvio della passata; i filtri correnti $f_{\min}$ e $f_{\max}$ sono fissati in quello stesso momento. Tutte le chiamate ricorsive di SAMPLE che seguono, inclusi i due richiami a livello $i-1$ nel passo ricorsivo, condividono la stessa NextCand e quindi lo stesso cursore di lettura: nessuna di esse riavvolge il nastro, né ne inizializza una propria copia, e il flusso di candidati viene così attraversato una sola volta, un elemento alla volta.

Ogni elemento x letto viene comunque confrontato con i filtri e, se necessario, utilizzato per aggiornare i contatori impiegati nel calcolo del loro rango. La procedura restituisce invece solamente gli elementi che soddisfano: $f_{\min} < x < f_{\max}$ cioè quelli appartenenti all'insieme C dei candidati.

Se il nastro termina senza che siano disponibili altri candidati, NextCand restituisce la «flag» End. In questo caso SAMPLE utilizza gli elementi fittizi necessari a completare la popolazione prevista dalla costruzione ricorsiva, tramite un'operazione di *padding*.

Algorithm 2 Costruzione ricorsiva del campione

```
 1: function SAMPLE(i, s)
[B]
 2:     if i = 0 then
 3:         X ← ∅
 4:         for j ← 1 to s do
 5:             x ← NextCand
 6:             if x = End then
 7:                 x ← *padding*
 8:             *inserisci x in X*
 9:         *ordina X in modo decrescente*
10:         return X
[R]
11:     X_1 ← SAMPLE(i − 1, s)
12:     X'_1 ← (X_1[2], X_1[4], ..., X_1[s])
[R]
13:     X_2 ← SAMPLE(i − 1, s)
14:     X'_2 ← (X_2[2], X_2[4], ..., X_2[s])
15:     return Merge(X'_1, X'_2)
```

Nel caso base la procedura legge un blocco di s candidati e li ordina, corrisponde esattamente al campione di livello 0 della Definizione 3.7. Nel caso ricorsivo, invece, vengono costruiti i due campioni di livello $i-1$ uno dopo l'altro e ciascuno viene assottigliato con il *thinning*, e solo alla fine i due sottocampioni vengono fusi insieme, tramite *merge*.

Osservazione 3.5. L'ordine delle operazioni non è casuale. Il primo campione deve essere assottigliato prima di costruire il secondo, così da mantenere in memoria soltanto $s/2$ elementi in attesa a ciascun livello della ricorsione. In questo modo l'occupazione di memoria resta coerente con il vincolo di spazio ricavato nella Sezione 3.3.2. Si noti inoltre che X'_1 e X'_2 non sono copie aggiuntive conservate accanto a X_1 e X_2: il thinning è da intendersi effettuato direttamente nel buffer originale, sovrascrivendo il campione di partenza con gli $s/2$ elementi selezionati, così che in memoria non compaiano mai simultaneamente le due versioni dello stesso sottocampione. Le celle non più necessarie diventano disponibili per le operazioni successive.

3.4.2 Singola passata

Una singola passata deve in realtà svolgere tre compiti insieme: tenere il conto del rango dei filtri correnti, costruire il campione a partire dai soli candidati, e infine scegliere la coppia di filtri da passare per la scansione successiva.

Per il primo compito bastano due contatori: uno conta quanti elementi dell'input superano $f_{\max}$, mentre l'altro quanti superano $f_{\min}$. Una volta giunti alla fine del nastro, questi due valori permettono di ricavare subito il rango esatto dei filtri, senza bisogno di una scansione dedicata.

Algorithm 3 Singola passata

function PASS($A, k, S, f_{\min}, f_{\max}, \bar{n}$)
  $s \leftarrow 2\left\lfloor \dfrac{S}{2\log_2 \bar{n}} \right\rfloor$
  $r \leftarrow \left\lceil \log_2\left(\dfrac{\bar{n}}{s}\right) \right\rceil$
  $q_{\max} \leftarrow 0$
  $q_{\min} \leftarrow 0$
  riavvolgi il nastro e inizializza NextCand
  $X \leftarrow$ SAMPLE(r, s)
  completa la scansione del nastro aggiornando $q_{\max}$ e $q_{\min}$
  if $f_{\max} = +\infty$ **then**
    $\rho_{\max} \leftarrow 0$
  else
    $\rho_{\max} \leftarrow q_{\max} + 1$
  if $f_{\min} = -\infty$ **then**
    $\rho_{\min} \leftarrow N + 1$
  else
    $\rho_{\min} \leftarrow q_{\min} + 1$
  if $\rho_{\max} = k$ **then**
    return (Found, $f_{\max}$)
  if $\rho_{\min} = k$ **then**
    return (Found, $f_{\min}$)
  $k_C \leftarrow k - \rho_{\max}$
  $u \leftarrow \left\lceil \dfrac{k_C}{2^r} \right\rceil - r$
  $v \leftarrow \left\lceil \dfrac{k_C}{2^r} \right\rceil$
  $f'_{\max} \leftarrow X[u]$
  $f'_{\min} \leftarrow X[v]$
  $\bar{n}' \leftarrow (2r-1)2^r$
  return (Continue, $f'_{\min}, f'_{\max}, \bar{n}'$)

Osservazione 3.6. Si noti che la procedura PASS riceve in ingresso $\bar{n}$ e non il numero effettivo n di candidati. Il valore $\bar{n}$ rappresenta infatti l'upper bound noto prima dell'inizio della passata ed è sufficiente per determinare i parametri s e r della costruzione del campione.

Inoltre la procedura può terminare restituendo due diversi stati:

- FOUND: indica che uno dei filtri ha rango esattamente pari a k e coincide quindi con l'elemento cercato. Insieme al flag viene restituito il valore del filtro.
- CONTINUE: indica che nessuno dei filtri coincide con $a_{(k)}$ e che la ricerca deve quindi proseguire. In questo caso vengono restituiti anche i nuovi filtri $f'_{\min}$, $f'_{\max}$ e l'upper bound $\bar{n}'$ da utilizzare nella passata successiva.

Durante la passata vengono mantenuti due contatori, $q_{\max}$ e $q_{\min}$, i quali tengono traccia rispettivamente del numero di elementi letti dal nastro maggiori di $f_{\max}$ e di $f_{\min}$. Entrambi vengono aggiornati durante la stessa scansione effettuata da NEXTCAND. Una volta completata la costruzione del campione, gli eventuali elementi ancora presenti sul nastro vengono semplicemente letti fino alla fine per completare i due conteggi.

3.4.3 Algoritmo multi-pass completo

L'algoritmo completo non fa altro che ripetere la procedura PASS finché l'upper bound sul numero di candidati non scende a $S - 2$ o meno, e ci si può ricondurre alla soluzione banale. Infatti, raggiunta quella soglia, tutti gli elementi ancora compresi tra i filtri possono finalmente essere mantenuti in memoria insieme ai filtri stessi, e costruire un ulteriore campione non è più necessario.

Prima di presentare lo pseudocodice completo conviene precisare come viene utilizzato il risultato restituito da PASS. Tale risultato viene raccolto nell'oggetto R, il cui campo $status$ può assumere due valori a seconda dello stato restituito dalla procedura:

- Se $R.status$ = FOUND, il campo $R.value$ contiene direttamente l'elemento di rango k e l'algoritmo termina.
- Se invece lo stato è CONTINUE, R contiene i nuovi filtri $f_{\min}$, $f_{\max}$ e il nuovo upper bound $\bar{n}$, che diventano i parametri della passata successiva.

Questa procedura viene eseguita finché $\bar{n} > S - 2$, poiché il numero effettivo di candidati soddisfa $n \leq \bar{n}$, quando questa condizione non è più verificata si ha certamente $n \leq S - 2$. A quel punto tutti i candidati rimasti possono essere mantenuti contemporaneamente in memoria insieme ai due filtri e non è quindi più necessario costruire un nuovo campione e la selezione può essere completata con un'ultima passata sul nastro.

Algorithm 4 Selezione multi-pass

```
1:  function MP-SELECT(A, k, S)
2:      f_min ← −∞
3:      f_max ← +∞
4:      n̄ ← N
5:      while n̄ > S − 2 do
6:          R ← PASS(A, k, S, f_min, f_max, n̄)
7:          if R.status = FOUND then
8:              return R.value
9:          f_min ← R.f_min
10:         f_max ← R.f_max
11:         n̄ ← R.n̄
12:     C ← ∅
13:     q_max ← 0
14:     q_min ← 0
15:     *riavvolgi il nastro*
16:     for each x ∈ A do
17:         if x > f_max then
18:             q_max ← q_max + 1
19:         if x > f_min then
20:             q_min ← q_min + 1
21:         if f_min < x < f_max then
22:             C ← C ∪ {x}
23:     if f_max = +∞ then
24:         ρ_max ← 0
25:     else
26:         ρ_max ← q_max + 1
27:     if f_min = −∞ then
28:         ρ_min ← N + 1
29:     else
30:         ρ_min ← q_min + 1
31:     if ρ_max = k then
32:         return f_max
33:     if ρ_min = k then
34:         return f_min
35:     k_C ← k − ρ_max
36:     *ordina C in modo decrescente*
37:     return C[k_C]
```

3.5 Correttezza e terminazione

La correttezza dell'intero algoritmo discende interamente sull'Invariante 3.1, se all'inizio di ogni passata la posizione di $a_{(k)}$ è garantita tra i due filtri correnti, dimostrare che l'algoritmo è corretto si riduce a tre verifiche:

- che l'invariante valga all'inizio dell'esecuzione
- che continui a valere ogni volta che i filtri vengono aggiornati
- che la procedura finale restituisca effettivamente l'elemento di rango k

Di seguito vi è la prova di questi tre punti:

Inizializzazione: Inizialmente i filtri sono fissati a $-\infty$ e $+\infty$, per cui vale banalmente:

$$-\infty \leq a_{(k)} \leq +\infty$$

L'invariante è dunque verificato ancora prima che l'algoritmo compia la prima operazione, qualunque sia la sequenza A in ingresso.

Mantenimento: Si supponga che l'invariante valga all'inizio di una generica passata. Se il rango di uno dei due filtri coincide proprio con k, PASS se ne accorge nella stessa passata e restituisce lo stato FOUND insieme a quel filtro: la selezione termina lì, correttamente.

Nel caso complementare, $a_{(k)}$ è strettamente compreso tra i due filtri, e tocca al campione della passata stabilire come restringere ulteriormente l'intervallo, restituendo lo stato CONTINUE. Indicato con $\rho(f_{\max})$ il rango del filtro superiore, la posizione di $a_{(k)}$ relativa alla sola popolazione dei candidati è, per la Definizione 3.11:

$$k_C = k - \rho(f_{\max})$$

Sia ora $X = (x_1, x_2, \ldots, x_s)$ il campione finale di livello r costruito durante la passata. Per la Definizione 3.9, ogni suo elemento x_j è legato ai limiti L_{rj} e M_{rj} e sono proprio questi limiti, applicati agli indici u e v ricavati nella Sezione 3.3.3, a garantire:

$$M_{ru} \leq k_C - 1 \leq L_{rv} \qquad \text{da cui} \qquad M_{ru} + 1 \leq k_C \leq L_{rv} + 1$$

La parte sinistra della disuguaglianza indica che, anche nel caso peggiore, sopra x_u non possono trovarsi più di $k_C - 1$ elementi, x_u non può quindi essere più piccolo dell'elemento di rango relativo k_C. La parte destra dice invece che sopra x_v se ne trovano almeno $k_C - 1$, e dunque x_v non può essere più grande dell'elemento cercato. È esattamente la situazione

nella parte (b) della Figura 3.4. Mettendo insieme, le due disuguaglianze si può collocare $a_{(k)}$ nella fascia evidenziata tra x_v e x_u:

$$x_v \leq a_{(k)} \leq x_u$$

e poiché la passata successiva $(p+1)$ adotta esattamente questi due valori come nuovi filtri:

$$f_{\min}^{(p+1)} = x_v, \qquad f_{\max}^{(p+1)} = x_u$$

l'Invariante 3.1 vale ancora al termine della passata.

Terminazione e selezione finale: Rimane infine da verificare che ogni passata riduca effettivamente l'upper bound sul numero di candidati. Dalla scelta dei nuovi filtri segue che, nella passata successiva, il numero effettivo di candidati n_{p+1} resta limitato dal nuovo upper bound $\bar{n}_{p+1}$ restituito da PASS:

$$n_{p+1} \leq \bar{n}_{p+1} = (2r_p - 1)2^{r_p}$$

che, con le scelte di s e r fissate nella Sezione 3.3.2 (indicando con pedice p la relativa passata), diventa:

$$\bar{n}_{p+1} = O\left(\frac{\bar{n}_p(\log \bar{n}_p)^2}{S}\right)$$

Per S sufficientemente grande rispetto a $(\log N)^2$, questa relazione garantisce che l'upper bound sul numero di candidati si riduca strettamente passata dopo passata, fino a far «crollare» tale bound a $S-2$ o meno (in un numero finito di passate), questo è il momento in cui, per costruzione di MP-SELECT, il ciclo principale si arresta.

A quel punto tutti i candidati rimasti possono essere mantenuti in memoria insieme ai due filtri, senza che sia più necessario costruire un ulteriore campione. Per l'Invariante 3.1, l'elemento $a_{(k)}$ può quindi ricadere in uno di questi due casi: o coincide con uno dei filtri oppure appartiene all'insieme C raccolto durante l'ultima passata. Nel primo caso viene individuato banalmente tramite il suo rango. Mentre nel secondo è sufficiente ordinare C e selezionare l'elemento di rango relativo k_C, ottenendo esattamente $a_{(k)}$.

Nelle ipotesi considerate, l'algoritmo termina quindi dopo un numero finito di passate e restituisce correttamente l'elemento di rango k.

Capitolo 4

Implementazione e analisi sperimentale

4.1 Scelte implementative

L'algoritmo è stato implementato in *Python* [11], principalmente per due motivi. Il primo è che permette di scrivere un codice «pulito» molto vicino allo pseudocodice del capitolo precedente, e quindi facile da confrontare con questo documento. Il secondo è la vasta disponibilità di librerie per l'analisi dei dati, che rende semplice ottenere grafici chiari, infatti per questo lavoro è stata utilizzata *Matplotlib* [12].

Il nastro di input è rappresentato da un flusso di testo che contiene un intero per riga. Gli elementi vengono letti uno alla volta, nell'ordine in cui arrivano, e non è mai possibile accedere direttamente a una posizione qualsiasi della sequenza. Per iniziare una nuova passata la «testina di lettura» viene riportata all'inizio, simulando proprio il riavvolgimento del nastro come nel modello teorico. In questo modo ogni passata dell'implementazione corrisponde a una scansione completa del nastro, e dunque poi il conteggio delle passate coinciderà con il numero di reinizializzazioni del nastro.

Maggiore attenzione richiede invece la memoria. Il parametro S indica il numero di celle logiche a disposizione dell'algoritmo, cioè quanti elementi possono essere mantenuti contemporaneamente, e non la memoria RAM occupata dall'interprete *Python*. Quest'ultima è molto maggiore, sia perché ogni oggetto occupa molti più byte di una singola cella, sia perché il programma deve gestire anche dati di servizio che non sono elementi dell'input (come ad esempio: $q_{\max}$ e $q_{\min}$, il riferimento al nastro, l'etichetta FOUND o CONTINUE restituita da ogni passata e i parametri s, r, k e $\bar{n}$). Si è quindi scelto di imporre il vincolo soltanto al modello e non all'intero processo, così da evitare complicazioni che non sono oggetto di questo studio. Durante una passata l'implementazione tiene in memoria soltanto il campione di lavoro di s elementi, un sottocampione di $s/2$ elementi per ciascuno degli r livelli e i due filtri. Queste strutture vengono create una sola volta, già della dimensione

prevista, e il *thinning* e il *merge* lavorano direttamente *in-place*, senza creare copie. Prima di ogni passata si controlla inoltre che lo spazio complessivo non superi S.

Come input sono state considerate permutazioni casuali degli interi da 1 a N, scritte una per riga sul nastro. L'algoritmo in realtà non dipende dal tipo dei valori: richiede soltanto che siano distinti e confrontabili tra loro, quindi funzionerebbe allo stesso modo anche con numeri reali. Gli interi sono però la scelta più comoda per la sperimentazione, perché permettono di generare in modo semplice e riproducibile sequenze di qualsiasi dimensione in cui la distinzione degli elementi è garantita per costruzione. Con valori in virgola mobile, invece, servirebbe un controllo aggiuntivo per evitare duplicati, ad esempio dovuti all'arrotondamento.

Infine anche il significato di S va precisato. La memoria del modello non è misurata in unità «commerciali», come byte, megabyte o gigabyte, ma in modo «posizionale»: una cella contiene esattamente un elemento dell'input, e S indica quindi quanti elementi possono essere mantenuti contemporaneamente. La dimensione dell'input N e la memoria S sono così espresse nella stessa unità, il numero di elementi, che per l'input corrisponde al numero di righe del file. Finché gli elementi appartengono a tipi *built-in* a dimensione limitata, come interi o numeri reali, il fatto che un valore occupi 2, 4 o 8 byte cambia la memoria fisica solo di un fattore costante e non altera il rapporto tra S e N. Il discorso sarebbe diverso se gli elementi fossero oggetti di dimensione variabile o molto grande, come stringhe lunghe o record complessi: in quel caso una singola cella potrebbe richiedere una quantità di memoria non controllata, e la definizione di S andrebbe rivista. Per gli input considerati in questa tesi, tuttavia, la misura posizionale descrive correttamente le risorse utilizzate.

4.2 Realizzazione dell'algoritmo

Il codice sviluppato per l'elaborato è raccolto in un repository GitHub pubblico, così che sia possibile consultarlo e riprodurre gli esperimenti presentati in questo capitolo. Il repository è disponibile al seguente indirizzo:

https://github.com/ill-barba/bsc-thesis-unipd-median-selection



ed è organizzato in tre parti: codice sorgente con il relativo algoritmo, i test per verificarne la robustezza e infine i file per eseguire gli esperimenti e ricavarne i grafici. Di seguito è riportato il *tree* del repository, con una breve descrizione di ciascun file:

```
bsc-thesis-unipd-median-selection/
|-- src/
|   `-- multipass.py              MP-SELECT e routine ausiliarie
|-- tests/
|   `-- test_multipass.py         test del codice
|-- experiments/
|   |-- metrics_multipass.py      conteggio di passate e candidati
|   |-- experiment_passes.py      esperimento sulle passate
|   |-- experiment_candidates.py  esperimento sui candidati
|   `-- results/                  dati CSV e grafici
|-- thesis-alessandro-barbacetto.pdf
|-- thesis-alessandro-barbacetto-ENG.pdf
|-- presentazione-per-discussione.pdf
|-- thesis-defense-presentation.pdf
|-- README.md
|-- LICENSE
`-- NOTICE
```

I codici descritti nel Capitolo 3 si trovano tutti in `multipass.py`, che non dipendono in alcun modo dal resto del progetto. I file della cartella `experiments/` lo utilizzano importandolo dall'esterno: generano gli input, eseguono l'algoritmo e ne misurano il comportamento, salvando i risultati in `results/`. Il file `README.md` spiega come installarne le dipendenze e lanciarne test ed esperimenti, mentre il funzionamento delle singole funzioni è documentato direttamente nel codice tramite *docstring*. Il software è distribuito con licenza *Apache 2.0*, riportata nel file `LICENSE`, e il file `NOTICE` indica chi ha realizzato il progetto. Nella cartella principale è infine disponibile anche una copia di questo elaborato.

4.3 Metodologia sperimentale

Gli esperimenti hanno lo scopo di verificare e osservare graficamente se il comportamento di MP-SELECT sia in linea con le garanzie ricavate nel Capitolo 3. L'analisi teorica aveva fornito due limiti superiori: il numero massimo di passate $P_{\max}$, che dipende solo da N e da S, e il numero massimo di candidati che rimangono in seguito a ciascuna passata. Questa parte sperimentale si propone quindi di accertare che l'implementazione rispetti tali garanzie e di osservare dove si collochino i valori effettivamente misurati.

Gli input sono costituiti da interi da 1 a N. Di questa sequenza si utilizza una permutazione casuale, creata da un generatore randomico inizializzato con un seme (*seed*) fissato,

a parità di seme la permutazione è deterministica, e ogni esecuzione può essere ripetuta in modo identico. Insieme a queste sono state considerate anche le sequenze ordinate in modo crescente e decrescente, che rappresentano due casi limite della disposizione dei dati sul nastro.

Le misure sono affidate al programma `metrics_multipass.py`, separato dall'algoritmo principale `multipass.py`. Il numero di passate P si ottiene contando i riavvolgimenti del nastro, mentre per osservare i candidati il processo che esegue una singola passata viene «avvolto» (*wrapped*) dall'esterno, così da registrarne lo stato e i nuovi filtri. Il numero effettivo di candidati viene poi determinato esaminando una copia completa dell'input, conservata solo per le misurazioni e quindi non soggetta al vincolo delle S celle di memoria dell'algoritmo. Lo stesso modulo calcola infine i bound teorici attesi applicando la ricorrenza su $\bar{n}$, senza esaminare alcun dato.

Le grandezze osservate sono la memoria S, che è il parametro che è fatto variare, il numero di passate P, confrontato con $P_{\max}$, e i candidati. Per questi ultimi si indica con n_p il numero di elementi compresi tra i filtri prodotti dalla passata p e con $\bar{n}_p$ il bound teorico, fissando inizialmente per convenzione $n_0 = \bar{n}_0 = N$.

Infine si osservi che l'input contiene esattamente gli interi da 1 a N, quindi l'elemento di rango k vale $N + 1 - k$ qualunque sia l'ordine sul nastro, e il risultato di MP-SELECT viene confrontato con questo valore, per valutarne la correttezza. Si verifica inoltre che il conteggio delle passate sia coerente con quelle eseguite e che dopo ogni passata valga $n_p \leq \bar{n}_p$. Su queste basi sono stati condotti due «esperimenti» approfonditi nelle prossime sezioni.

4.4 Memoria e numero di passate

La prima analisi riguarda il *trade-off* principale del modello di Munro e Paterson, ovvero quante volte occorre scorrere il nastro (il numero di passate) in funzione della memoria disponibile. Sono state considerate quattro dimensioni dell'input crescenti nella taglia di un fattore 10, rispettivamente: $N = 10^3, 10^4, 10^5, 10^6$, cercando in ciascuno di essi la mediana $k = \lceil N/2 \rceil$ su una permutazione casuale (di seed 2026).

Per ciascuno dei quattro valori di N, la memoria S non è stata fatta variare su valori interi, ma su una griglia geometrica, nella quale ogni valore si ottiene moltiplicando il precedente per $2^{1/4} \simeq 1{,}189$, cioè aumentandolo di circa il 19% (arrotondando il risultato all'intero più vicino). Dunque dopo quattro incrementi la memoria risulta essere raddoppiata. Con questa scelta gli incrementi sono piccoli in valore assoluto quando la memoria è ridotta e diventano via via più grandi al crescere di S. In questo modo è possibile osser-

vare con un numero contenuto di esecuzioni un intervallo di memoria molto esteso, che comincia da poche centinaia di celle fino all'intero input.

Il punto di partenza delle ascisse dipende dalla dimensione dell'input N fissata ed è indicato con $S_{\min}(N)$ il più piccolo valore intero di memoria per cui la ricorrenza dell'algoritmo riesce a ridurre il limite sui candidati fino alla selezione finale, per quella taglia dell'input. Per i quattro valori di N considerati, il vincolo fondamentale riguarda la seconda passata, se il campione non è abbastanza grande da far scendere il numero di livelli rispetto al precedente, il limite resta invariato e l'algoritmo non migliora. Indicando con r_1 il numero di livelli della prima passata e con $\bar{n}_1 = (2r_1 - 1)2^{r_1}$ il limite che essa restituisce, si trova:

$$S_{\min}(N) = \left\lceil 2(2r_1 - 1)\log_2 \bar{n}_1 \right\rceil,$$

da cui si può calcolare in modo esplicito per ogni N:

- $N = 10^3$: $S_{\min} = \lceil 22 \log_2 704 \rceil = 209$;
- $N = 10^4$: $S_{\min} = \lceil 34 \log_2 8704 \rceil = 445$;
- $N = 10^5$: $S_{\min} = \lceil 46 \log_2 94208 \rceil = 761$;
- $N = 10^6$: $S_{\min} = \lceil 58 \log_2 950272 \rceil = 1152$.

Questi valori corrispondono «a occhio» ai punti di inizio delle varie curve nella Figura 4.1. Poi a partire da questi valori la memoria cresce secondo l'incremento:

$$S_j = \text{round}\left(S_{\min}(N) \cdot 2^{j/4}\right), \qquad j = 0, 1, 2, \ldots$$

finché $S_j < N + 2$. Quando si raggiunge infine $S = N + 2$, la memoria è sufficiente a contenere l'intero input insieme ai due filtri e l'algoritmo termina.

Dunque, nel grafico l'asse orizzontale S è in scala logaritmica. La memoria varia infatti di quasi quattro ordini di grandezza e, se avessi adottato un asse lineare, la parte iniziale delle curve, dove il numero di passate cambia più rapidamente, sarebbe stata «schiacciata» verso l'origine. Al contrario con una scala logaritmica, i valori crescono sempre della stessa percentuale e risultano equispaziati.

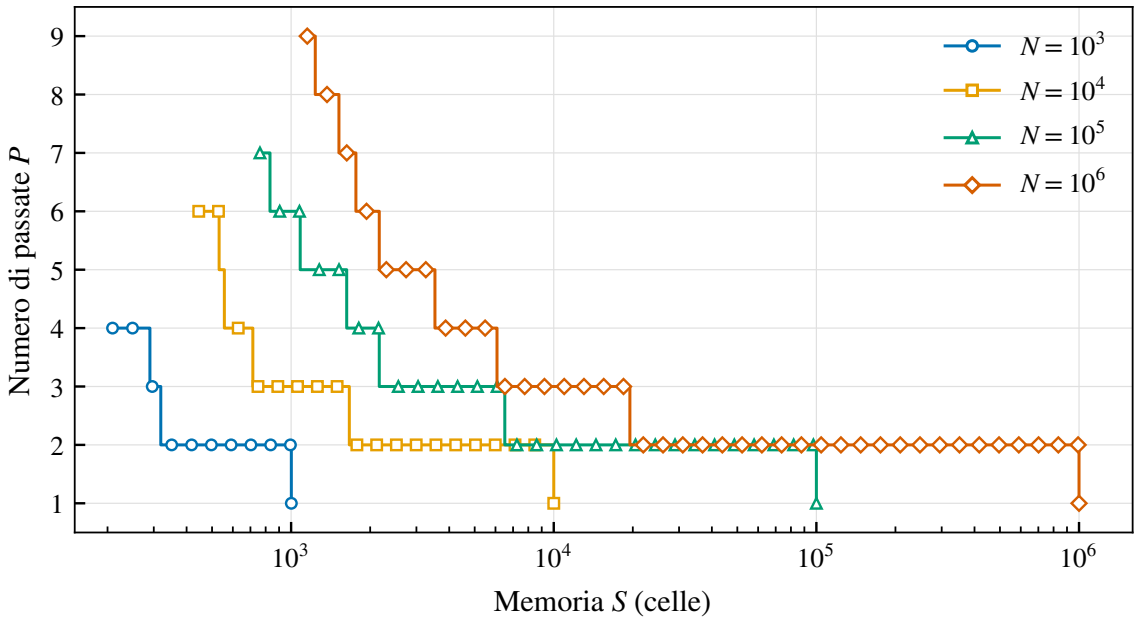


Figura 4.1: Numero di passate in funzione della memoria S (scala logaritmica), per $N = 10^3, 10^4, 10^5, 10^6$ e $k = \lceil N/2 \rceil$. Le curve «a gradini» rappresentano $P_{\max}$, calcolato dalla ricorrenza su $\bar{n}$. Mentre i marcatori vuoti sono i valori osservati di P.

Nella Figura 4.1 la prima cosa che si osserva è che tutti i marcatori cadono esattamente sulle rispettive curve «a gradini», dunque il numero di passate misurato eseguendo l'algoritmo coincide sempre con quello previsto dalla ricorrenza. Però va osservato un aspetto: $P_{\max}$ è il numero massimo di passate previsto dalla teoria, e P quindi può risultare inferiore solo se una passata termina in anticipo restituendo FOUND. Il fatto che nessun marcatore cada sotto la propria linea indica che, sugli input di interesse, non è mai accaduto. Segue che, in assenza di terminazioni anticipate il numero di passate è fissato dai parametri della ricorrenza.

Per quanto riguarda l'andamento del grafico, complessivamente è quello atteso. Infatti, nessuna curva «risale», cioè all'aumentare della memoria il numero di passate non cresce mai, ma si riduce con un ritmo in modo discreto, a gradini. La ragione di tutto questo sta nella ricorrenza stessa. I parametri s e r di ogni passata sono dei numeri interi, per cui un piccolo aumento di S può lasciarli invariati, e con essi il limite sui candidati. Anche quando il limite di una passata si riduce, il numero complessivo di passate diminuisce soltanto se la riduzione è tale da portare $\bar{n}$ sotto $S - 2$ con una passata di anticipo. Ne risulta che $P_{\max}$ resta costante su interi intervalli di memoria per poi crollare in corrispondenza di determinate soglie, sempre di una sola unità alla volta.

Più interessante, invece, è la larghezza dei gradini, che in scala logaritmica indica di quante volte la memoria deve crescere per risparmiare una passata. I gradini sono in generale più «stretti» vicino alla memoria minima $S_{min}(N)$ e si «allargano» rapidamente

spostandosi verso destra, fino a quello a due passate ($P = 2$), che da solo occupa gran parte del grafico. Questo comportamento si spiega grazie alla garanzia di diminuimento in ogni passata, ricavata nella Sezione 3.5, la quale divide il limite sui candidati per un fattore almeno dell'ordine di $S/(\log N)^2$, e dopo le passate di campionamento il limite deve essere sceso abbastanza da poter essere contenuto in memoria. Ne segue che, a meno di fattori logaritmici, la memoria necessaria per concludere in P passate è dell'ordine di $N^{1/P}$, che diventa $\sqrt{N}$ nel caso di interesse con due passate, la radice cubica per tre, e così via [3]. Quando le passate sono molte (valori di N maggiori), le radici successive di N sono vicine tra loro e basta un piccolo aumento di memoria per risparmiarne una; quando sono poche, invece, sono molto distanti, e tra $\sqrt{N}$ e N stesso il divario è enorme.

La stessa relazione può essere letta anche nel verso opposto, per stimare il numero di passate fissando la memoria. Ponendo $S = N^{1/P}$ e passando ai logaritmi si ottiene $\log S = \frac{\log N}{P}$, da cui:

$$P \approx \frac{\log N}{\log S}$$

La stima trascura i fattori logaritmici, per cui va intesa come intuizione dell'andamento. Ne mette però in evidenza l'aspetto essenziale: a memoria fissata, il numero di passate dipende da $\log N$ e non da N. Viceversa, fissata la taglia dell'input il numero di passate è inversamente proporzionale al logaritmo della memoria.

Il confronto tra le quattro curve mostra inoltre quanto poco l'algoritmo risenta della dimensione dell'input. La memoria minima cresce molto più lentamente di N, infatti per un milione di elementi bastano poco più di mille celle, circa un millesimo dell'input, al prezzo di nove passate. Allo stesso modo, a parità di memoria, un input dieci volte più grande richiede soltanto qualche passata in più, e quando la memoria è sufficientemente ampia la differenza svanisce del tutto: tra circa 20000 e 100000 celle le curve di $N = 10^5$ e $N = 10^6$ si sovrappongono sul gradino a due passate, pur riferendosi a input che differiscono di un fattore dieci.

Infine, poiché $P_{\max}$ dipende soltanto da N e da S, il grafico può essere letto in entrambi i sensi. Fissata la memoria disponibile, indica quante passate saranno necessarie, e rispettivamente fissato il numero di passate che si è disposti a eseguire, indica quanta memoria è necessaria. In entrambi i casi la risposta è nota prima ancora di leggere e analizzare i dati grazie alla garanzia deterministica ricavata nel Capitolo 3, che è indipendente dall'ordine degli elementi e dal rango cercato.

4.5 Candidati effettivi e limite teorico

Nella sezione precedente il numero di passate è determinato dalla ricorrenza su $\bar{n}$, la quale non dipende dai dati, ma questo non dice quanti candidati restino davvero tra i filtri dopo ogni passata, ma ne fornisce solo un bound. Questa seconda analisi è incentrata proprio su questo. Per ciascuna passata p si confrontano il numero effettivo di candidati rimasti n_p, contato direttamente sull'input, e il corrispondente limite teorico $\bar{n}_p$, che dipende soltanto dal numero di livelli r_p del campione costruito su quella passata. La base teorica ci garantisce che valga sempre $n_p \leq \bar{n}_p$, ma non dice di quanto i due valori siano distanti tra loro.

Il confronto è interessante perché è proprio il bound, e non il numero effettivo di candidati, a guidare l'algoritmo, infatti da $\bar{n}_p$ dipendono i parametri della passata successiva e il momento in cui si passa alla selezione finale. Il limite per questo motivo è molto conservativo e il suo valore si ottiene supponendo che i due nuovi filtri occupino contemporaneamente la posizione più sfavorevole consentita dalla Definizione 3.9, cioè per il filtro superiore la più alta possibile e per quello inferiore la più bassa. Questo perché il limite deve valere per qualunque input, e l'analisi serve a capire quanto ne risulti pessimistico e cosa ne comporti per il funzionamento dell'algoritmo.

Per questo secondo esperimento sono state considerate due dimensioni della taglia dell'input, rispettivamente $N = 10^4$ e $N = 10^5$, cercando in entrambi i casi la mediana $k = \lceil N/2 \rceil$. Rispetto a prima, per ciascuna configurazione sono state utilizzate 30 permutazioni casuali, generate con semi da 1 a 30, per semplicità. Ne risulta che il limite teorico $\bar{n}_p$ rimane invariato a parità di N e S, come accennato precedentemente. Mentre il numero effettivo di candidati n_p può cambiare in funzione della disposizione degli elementi sul nastro. Le diverse permutazioni permettono dunque di osservarne sia il valore medio sia la variabilità. Anche in questo caso la memoria S è stata fatta variare su una griglia geometrica, partendo dai valori $S_{\min}(N)$ già ricavati nella sezione scorsa. La griglia prosegue fino all'ultimo valore di S inferiore a $N + 2$, soglia alla quale la memoria diventa sufficiente a contenere l'intero input insieme ai due filtri.

Nel primo pannello (a), Figura 4.2a, vengono raccolti i risultati di tutte le configurazioni considerate e se ne fa una media. Per ogni passata p si riporta sull'asse orizzontale il limite teorico $\bar{n}_p$ e su quello verticale la media dei candidati effettivi n_p sulle 30 permutazioni. Ogni marcatore vuoto rappresenta quindi una terna (N, S, p). Entrambi gli assi sono in scala logaritmica, poiché il numero di candidati varia su diversi ordini di grandezza. Sono inoltre riportate due rette tratteggiate che forniscono una «guida» visiva. Quella rossa rappresenta il limite teorico $n_p = \bar{n}_p$, mentre quella blu, $n_p = \bar{n}_p/4$, in seguito verrà osservato che l'interpolazione dei marcatori giacerebbe proprio sulla guida a $n_p = \bar{n}_p/2$.

Il secondo pannello (b), Figura 4.2b, considera invece una sola configurazione, con un vettore di input di taglia $N = 10^5$ e una memoria $S = 761$, scelta sempre guardando $S_{min}(N)$. In questo caso l'asse orizzontale rappresenta il numero della passata p corrente, mentre quello verticale, ancora in scala logaritmica, indica il numero di candidati risultanti. Qui viene confrontato il limite teorico $\bar{n}_p$, rappresentato dalla curva rossa tratteggiata, e la media dei candidati effettivi n_p sulle 30 permutazioni, rappresentata con la curva verde. Il punto $p = 0$ rappresenta la situazione iniziale, in cui $n_0 = \bar{n}_0 = N$, mentre la linea orizzontale blu indica la soglia $S - 2$, al di sotto della quale è possibile procedere con la selezione finale.

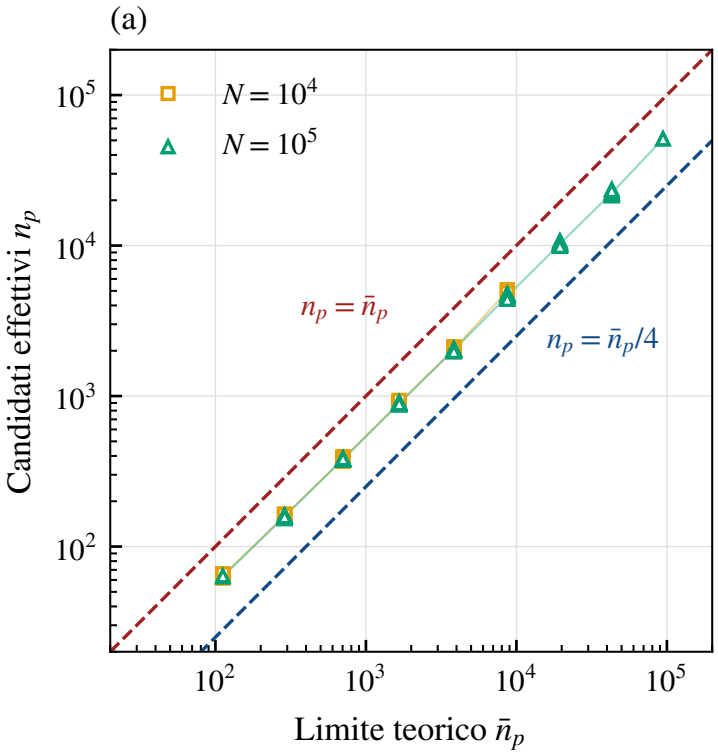
Osservando la Figura 4.2a, si nota che i punti si dispongono su colonne verticali, ciascuna associata a un diverso numero di livelli r_p del campione. Questo accade perché il limite teorico $\bar{n}_p$ dipende soltanto da r_p, per cui configurazioni con valori di memoria e passate differenti possono condividere lo stesso limite.

Osservando la Figura 4.2b, si nota che il numero effettivo di candidati diminuisce progressivamente insieme al limite teorico, mantenendosi sempre al di sotto di esso, tranne nella situazione iniziale in cui i due valori coincidono. Anche le «barre» che rappresentano il minimo e il massimo sulle permutazioni rimangono entro il limite, mostrando che la garanzia viene rispettata non soltanto dalla media, ma da tutte le esecuzioni considerate.

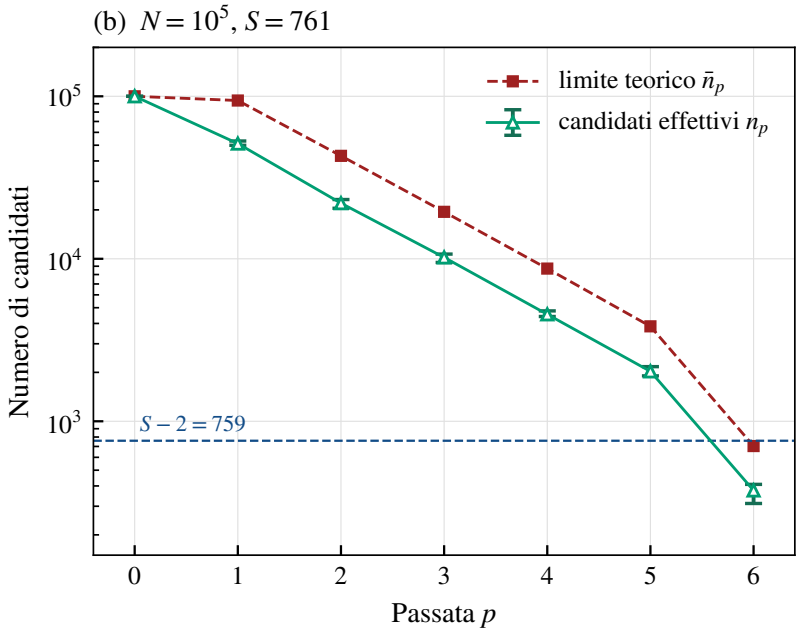
Il primo risultato, osservabile nella Figura 4.2a, è garantito dalla teoria: il limite è sempre rispettato, sia dalle medie sia dalle singole esecuzioni. Il margine è inoltre molto ampio, perché nelle passate di campionamento nemmeno il caso più sfavorevole osservato raggiunge i tre quarti di $\bar{n}_p$. Più interessante è la distribuzione dei punti, su circa tre ordini di grandezza, questi sembrano sempre giacere circa a metà strada tra le due linee guida tratteggiate. Si può concludere quindi che i candidati effettivi sono quindi, in media, poco più della metà del limite teorico.

Ancora più regolare è il comportamento all'interno delle «colonne immaginarie» citate precedentemente. Infatti i punti ottenuti con memorie diverse, in passate diverse e per entrambe le dimensioni dell'input si trovano praticamente alla stessa ordinata. A parità di numero di livelli, la prima passata e l'ultima, con poca o con molta memoria, lasciano in media la stessa frazione di candidati. Una possibile spiegazione intuitiva di questo comportamento viene dal modo in cui il limite è definito. Esso comprende sia la distanza che normalmente separa i due filtri sia il margine necessario a coprire le loro posizioni più sfavorevoli, due contributi semanticamente simili.

Altre simulazioni, non rappresentate nei grafici, permettono di considerare anche altri ranghi e gli input ordinati. Il bound viene sempre rispettato, ma il margine cambia. Il rango $k = \lceil N/10 \rceil$ si comporta in modo simile alla mediana, mentre per altri ranghi «estremi»



(a) Media di n_p in funzione di $\bar{n}_p$ per ogni terna (N, S, p), in scala logaritmica. Le linee tratteggiate rappresentano il limite $n_p = \bar{n}_p$ in rosso e il riferimento $n_p = \bar{n}_p/4$ in blu.



(b) Limite $\bar{n}_p$ e media di n_p, con barre tra minimo e massimo, passata per passata per $N = 10^5$ e $S = 761$. La retta tratteggiata indica la soglia $S - 2$.

Figura 4.2

i candidati sono sensibilmente meno numerosi, soprattutto per $k = 1$. In questi casi si osserva che uno dei due filtri tende a restare fisso ad un valore di sentinella, ad esempio infinito, (per $k = 1$, infatti, il filtro superiore non viene mai aggiornato). Questo potrebbe spiegare il minor numero di candidati osservato, l'intervallo di ricerca è infatti delimitato da un solo lato, mentre il limite teorico è calcolato considerando il caso più sfavorevole per entrambi i filtri. Infine sugli input già ordinati si sono osservate delle terminazioni anticipate, ad esempio un'esecuzione con $k = N$, conclusa con FOUND, e quindi con un numero di passate inferiore a $P_{\max}$.

Passando alla Figura 4.2b, si può osservare come il margine tra candidati effettivi e limite teorico si sviluppi nel corso delle singole passate. La distanza tra le due curve si apre già a partire dalla prima passata e rimane poi pressoché costante. I candidati effettivi, invece, diminuiscono già a poco più della metà del limite, cioè a circa metà dell'input. La garanzia iniziale risulta quindi particolarmente pessimistica, ma questo aspetto è legato alla memoria minima scelta. Infatti nelle configurazioni con più memoria, già la prima passata riduce sensibilmente anche il limite teorico.

Le barre nei marcatori del pannello (b) mostrano invece quanto le singole permutazioni si discostino dalla media, si può notare come siano appena percettibili nelle prime passate e si allargano soprattutto nell'ultima, quando i candidati sono ormai poche centinaia. È ragionevole pensare a un effetto di compensazione della media. Per l'appunto su molti elementi le irregolarità della singola permutazione tendono a compensarsi, mentre su poche centinaia di candidati anche qualche elemento in più o in meno pesa maggiormente nella media totale. Si può dedurre quindi che il numero di candidati rimasti dopo una passata è tanto più stabile quanto più grande era il limite.

Un altro aspetto interessante della Figura 4.2b riguarda la soglia $S - 2$. L'algoritmo continua il campionamento finché $\bar{n}_p > S - 2$, a decidere il passaggio alla selezione finale è quindi il bound, e non il numero effettivo di candidati. Nella figura le due curve attraversano la soglia nella stessa passata, ma non è sempre così in generale. Quando il bound della penultima passata supera la soglia, i candidati effettivi possono trovarsi già sotto di essa una passata prima, percependo quindi un «rallentamento». Il fatto che accada o meno dipende, come per i «gradini» della Sezione 4.4, da dove cade la soglia rispetto ai valori discreti che il limite può assumere. In questi casi la passata aggiuntiva è dovuta soltanto alla prudenza del limite, e il suo costo corrisponde comunque a una scansione completa del nastro.

Un'altra conseguenza di questo margine riguarda la memoria utilizzata nella selezione finale. Poiché il limite termina spesso ben al di sotto della soglia $S-2$ e raramente coincide con essa, e il numero di candidati effettivi è a sua volta inferiore al limite, una parte delle

celle disponibili rimane inutilizzata. Nel caso della Figura 4.2b, i candidati occupano circa metà delle celle disponibili, dunque la memoria riservata alla passata finale è quindi dimensionata sul caso peggiore e, nelle esecuzioni osservate, resta in buona parte vuota.

L'algoritmo, d'altro conto, non potrebbe basare direttamente le proprie decisioni sul numero effettivo di candidati, perché al termine di una passata conosce i nuovi filtri, ma non quanti elementi siano compresi tra essi, e per saperlo dovrebbe scorrere nuovamente il nastro. Il limite, invece, è noto in anticipo e garantisce in modo deterministico il rispetto del vincolo della memoria.

4.6 Considerazioni finali e limiti

Considerati insieme, i due esperimenti mostrano due facce della stessa scelta. MP-SELECT usa i dati effettivi per scegliere i nuovi filtri, ma i parametri di ogni campione e il momento in cui si interrompe il campionamento dipendono soltanto dal limite $\bar{n}$. L'unica eccezione è il caso, osservato raramente, in cui un filtro coincida con l'elemento cercato. Da un lato questo rende prevedibile il numero massimo di passate, che segue la ricorrenza teorica e si riduce a «gradini» all'aumentare della memoria. Dall'altro, lo stesso limite si rivela molto conservativo, infatti nelle configurazioni considerate sovrastima i candidati effettivi e in alcuni casi costringe l'algoritmo a una passata che i dati, da soli, non avrebbero richiesto. Prevedibilità e prudenza sono quindi, rispettivamente, il vantaggio e lo svantaggio della stessa garanzia.

Lo studio presenta inoltre alcuni limiti. Gli input sono sintetici, permutazioni casuali e sequenze ordinate di interi distinti, mentre dati reali possono avere distribuzioni diverse e contenere talvolta valori ripetuti, che il modello esclude per ipotesi. La memoria, poi, è misurata in celle logiche, secondo il modello descritto nella Sezione 4.1, e non coincide con la RAM effettivamente occupata da *Python*. Infine, l'analisi misura il costo dell'algoritmo in numero di passate, come nel modello teorico, e non in tempi di esecuzione, cioè quanto costi davvero una passata, che per l'appunto, dipende dal supporto su cui sono salvati i dati e dall'hardware che stabilisce la velocità con cui possono essere riletti.

Un possibile punto di riflessione per uno sviluppo futuro riguarda proprio il margine osservato tra il limite teorico e il numero effettivo di candidati, che nella selezione finale lascia spesso inutilizzata una parte della memoria disponibile. Si potrebbe valutare una variante euristica dell'algoritmo che, quando $\bar{n}$ si trova poco al di sopra della soglia $S - 2$, tenti di «anticipare» la selezione finale sulla base di una stima del numero effettivo di candidati. Durante questa scansione gli elementi verrebbero raccolti in memoria soltanto fino alla capacità consentita e se i candidati fossero effettivamente al più $S - 2$, la selezione

potrebbe concludersi senza ulteriori campionamenti. Mentre in caso contrario, sarebbe necessario riprendere il procedimento ordinario con una passata successiva. Una strategia di questo tipo manterrebbe il vincolo di memoria, ma introdurrebbe il rischio di effettuare una scansione aggiuntiva senza ottenere alcun vantaggio, perdendo tempo di esecuzione. Sarebbe quindi interessante studiare in quali condizioni il tentativo risulti conveniente, confrontando il numero di passate risparmiate con quelle eventualmente aggiunte, in relazione ai check «a vuoto» se i candidati rimasti non dovessero «fittare» la memoria a disposizione. L'obiettivo non sarebbe utilizzare necessariamente tutte le celle disponibili, ma sfruttarle al meglio.

Proprio su quest'ultimo aspetto calca più la mano il capitolo successivo. Infatti nel Capitolo 5 l'algoritmo viene collocato in uno scenario reale, in cui i dati vengono acquisiti in modo sequenziale e registrati su una memoria che può essere riletta più volte, mentre la memoria di lavoro è ridotta e il tempo a disposizione per l'elaborazione è limitato, ed è cruciale rispettarlo.

Capitolo 5

Esempio applicativo

5.1 Scenario applicativo

Come applicazione dell'algoritmo multi-pass di Munro e Paterson si considera un sistema embedded installato su una monoposto di Formula 1 durante una sessione di qualifica. La vettura sta effettuando uno o più giri di preparazione con l'obiettivo di portare gli pneumatici nella finestra termica corretta, in modo da disporre della massima aderenza per affrontare un giro veloce [5].

Durante ciascun giro il dispositivo acquisisce ad alta frequenza la temperatura degli pneumatici e la registra progressivamente su una memoria non volatile. Il goal del sistema è di stabilire prima del passaggio successivo sul traguardo, se le condizioni termiche permettano di affrontare il giro seguente come *push lap*, oppure se sia necessario un ulteriore giro di preparazione.

Per mantenere coerente il miniworld si formulano le seguenti ipotesi:

1. I quattro pneumatici presentano un comportamento termico equivalente, quindi non ci saranno ruote con temperature differenti e dunque si utilizzerà una singola sequenza di temperature.

2. Per ogni giro di preparazione la finestra di acquisizione delle temperature inizia al passaggio sul traguardo e termina con l'ingresso nell'ultimo microsettore del terzo settore.

3. Al termine della finestra di acquisizione la sequenza utilizzata per la decisione è completa, eventuali campioni acquisiti successivamente non appartengono alla sequenza analizzata per quel giro.

4. La memoria non volatile è sufficientemente grande da contenere tutti i campioni acquisiti, mentre la RAM disponibile per il sistema embedded è molto più piccola

e viene modellata mediante le S celle di memoria (nel Capitolo 3); i dati registrati possono essere riletti più volte in modo sequenziale.

5. Esiste una finestra termica di riferimento $[T_{\min}, T_{\max}]$ all'interno della quale lo pneumatico è considerato in una condizione favorevole per iniziare un giro veloce.

6. Si assume che nella successione dei giri di preparazione, esista sempre un giro la cui mediana appartenga alla finestra $[T_{\min}, T_{\max}]$, escludendo così il caso in cui la finestra venga «saltata» tra due giri consecutivi senza che nessuna delle due mediane la rilevi.

7. In accordo con il modello teorico adottato nel Capitolo 3, i campioni vengono trattati come elementi distinti. Eventuali misurazioni di temperatura coincidenti possono essere discriminate mediante il loro ordine di acquisizione.

8. Il calcolo della mediana e la successiva comunicazione al pilota devono essere completati durante la percorrenza dell'ultimo microsettore, quindi prima dell'inizio del giro successivo.

Dalle ipotesi 2 e 3, acquisizione ed elaborazione avvengono dunque in due momenti distinti: durante la prima parte del giro il sistema si limita a misurare e memorizzare le temperature, mentre all'ingresso nell'ultimo microsettore l'acquisizione si interrompe e la sequenza appena raccolta diventa l'input dell'algoritmo multi-pass, che viene eseguito su di essa prima dell'inizio del giro successivo. Al termine dell'analisi il sistema comunica al pilota un'indicazione binaria:

- PUSH, se il giro successivo può essere affrontato come giro veloce
- PREP, se è invece necessario un ulteriore giro di preparazione

La memoria non volatile svolge in questo scenario il ruolo del nastro di input del modello computazionale, mentre la RAM disponibile corrisponde alle S celle di memoria di lavoro.

Osservazione 5.1 (Finestra temporale reale)**.** Per dare un ordine di grandezza del tempo disponibile per l'ipotesi 8, si può prendere come riferimento il giro della pole position realizzato da Charles Leclerc a Monza nel 2022, completato in 1:20.161, con il terzo settore percorso in circa 26.553 secondi [8, 9].

Assumendo una suddivisione del terzo settore in sette microsettori e considerandone, per semplicità, la durata media, si ottiene:

$$\frac{26.553}{7} \approx 3.79 \text{ s}$$

L'ultimo microsettore è qui idealmente fatto iniziare nella zona della Curva Alboreto e terminare sulla linea del traguardo, il valore di 3.79 secondi dà un ordine di grandezza della finestra temporale a disposizione.

Le operazioni eseguite sui campioni già presenti in RAM, come confronti, *thinning* e *merge*, avvengono su scale temporali molto inferiori al secondo. Inoltre l'esecuzione complessiva dell'algoritmo comprende anche più riletture sequenziali della memoria, e il suo tempo effettivo andrebbe quindi verificato sperimentalmente. In questo miniworld si assume, però, per l'Ipotesi 8, che una finestra dell'ordine di 3-4 secondi sia ampiamente sufficiente a completare l'analisi e a comunicare la decisione al pilota.

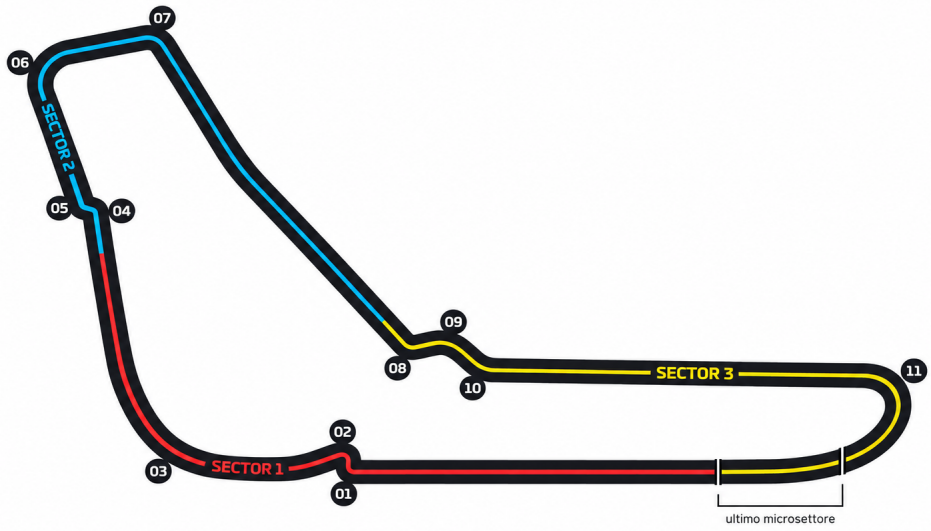


Figura 5.1: Qui sopra vi è uno schema semplificato del circuito di Monza [10] con in evidenza il tratto dell'ultimo microsettore. La finestra di acquisizione delle temperature termina in corrispondenza di questa porzione, e inizia l'elaborazione.

5.2 Modello semplificato

Rimanendo fedeli all'ipotesi 1, il sistema viene dunque rappresentato con un'unica sequenza di temperature per ciascun giro di preparazione. Sia:

$$T_j = (t_{j,1}, t_{j,2}, \ldots, t_{j,N_j})$$

la sequenza dei campioni di temperatura acquisiti durante il giro j, dove N_j è il numero di misurazioni effettuate nella finestra secondo l'ipotesi 2 nel giro j, cioè dal passaggio sul traguardo all'ingresso nell'ultimo microsettore del terzo settore.

Osservazione 5.2 (campioni per giro)**.** Il numero di campioni N_j non è necessariamente uguale per tutti i giri. Nonostante la frequenza di campionamento sia sempre costante, da giro a giro può, invece, cambiare la durata della finestra di acquisizione in funzione del relativo tempo di percorrenza. Di conseguenza, anche il numero complessivo di misurazioni raccolte può cambiare.

Inoltre, all'interno della stessa finestra due campioni possono presentare lo stesso valore di temperatura. Tali misurazioni non vengono eliminate, poiché ciò modificherebbe la distribuzione dei dati e quindi potenzialmente anche la mediana. In accordo con l'ipotesi 7, eventuali valori coincidenti vengono invece considerati distinti attraverso il loro ordine di acquisizione.

Su T_j successivamente si calcola la mediana: $m_j = \text{median}(T_j)$, che corrisponde a un problema di selezione con rango cercato pari a: $k_j = \left\lceil \frac{N_j}{2} \right\rceil$ risolto direttamente con l'algoritmo multi-pass:

$$m_j = \text{MP-SELECT}(T_j, k_j, S)$$

Nota che in questo esempio la scelta della mediana (piuttosto che una media dei campionamenti) non è casuale, infatti durante un giro possono verificarsi delle variazioni brevi di temperatura o dei singoli campioni anomali. Queste variazioni sono principalmente dovute, ad esempio, a un'uscita momentanea dalla traiettoria ideale o a una zona locale dell'asfalto differente rispetto al resto del circuito oppure ad un errore isolato di misurazione. Questi valori possono influenzare e «falsare» sensibilmente la media anche se riguardano solo una piccola parte delle misurazioni, mentre la mediana, dipendendo principalmente dalla posizione dei campioni ordinati, fornisce un indicatore più stabile e robusto a questi cambiamenti.

5.3 Finestra termica di riferimento

La decisione del sistema si basa sul confronto tra m_j e la finestra termica di riferimento definita nell'ipotesi 5. I due estremi $T_{\min}$ e $T_{\max}$ non sono determinati dall'algoritmo, ma costituiscono parametri applicativi forniti in ingresso al sistema.

Il loro valore dipende dalle caratteristiche dello pneumatico e dalle condizioni operative considerate. In particolare, la finestra termica può variare in funzione della mescola utilizzata o, più in generale, delle condizioni che influenzano l'interazione tra pneumatico e pista, ad esempio il livello di aderenza disponibile e le caratteristiche abrasive dell'asfalto [5]. Per questo motivo $T_{\min}$ e $T_{\max}$ devono essere stabiliti a priori sulla base dei dati sperimentali disponibili per lo specifico scenario considerato.

Una volta calcolata la mediana m_j del giro j-esimo, si distinguono tre casi:

- se $m_j < T_{\min}$, la temperatura osservata è ancora inferiore alla finestra di riferimento, dunque il sistema comunica PREP
- se $T_{\min} \leq m_j \leq T_{\max}$, la mediana del giro ricade nella finestra considerata ottimale, dunque il sistema comunica PUSH
- se $m_j > T_{\max}$, la temperatura osservata supera la finestra di riferimento, dunque il sistema comunica PREP

In questo modello considerato si assume inoltre che, durante una normale sequenza di giri di preparazione, la temperatura degli pneumatici aumenti in modo progressivo, o al più rimanga temporaneamente stabile. Di conseguenza anche le mediane m_j dei giri successivi seguono un andamento non decrescente. In altre parole non si hanno picchi improvvisi di temperature tra giri consecutivi, o «crolli» repentini, che potrebbero far saltare la finestra ottimale e renderla irraggiungibile.

Formalmente quindi si assume inoltre che l'aumento tra due giri consecutivi sia sufficientemente graduale da non permettere il passaggio diretto da una condizione:

$$m_j < T_{\min}$$

a una condizione:

$$m_{j+1} > T_{\max}$$

In altre parole, si assume che non possano esistere due giri consecutivi tali che $m_j < T_{\min}$ e $m_{j+1} > T_{\max}$.

Riassumendo, questa ipotesi descrive il funzionamento in condizioni normali di preparazione. Eventi anomali, come un'uscita di pista, un'interruzione del giro o il ritiro della vettura, non vengono considerati nel modello base.

5.4 Individuazione dei giri di preparazione

Data una successione di giri di preparazione, l'obiettivo è individuare il primo giro per cui si verificano le condizioni favorevoli per un giro veloce. Si definisce quindi:

$$j^* = \min \left\{ j \;\middle|\; T_{\min} \leq m_j \leq T_{\max} \right\}$$

Il giro indicizzato con j^* è il primo al termine del quale il sistema comunica PUSH, quindi il giro successivo può essere affrontato come possibile *push lap*. Per ciascun giro j la decisione si ottiene dunque applicando l'algoritmo multi-pass alla sequenza T_j per ricavarne la mediana $m_j = \text{MP-SELECT}(T_j, k_j, S)$, e confrontandola infine con la finestra termica per determinare se comunicare PUSH oppure PREP.

Osservazione 5.3 (Esistenza del giro ottimale)**.** L'esistenza di un giro j^* che soddisfi la definizione precedente non costituisce una garanzia assoluta, ma discende dalle ipotesi del modello semplificato. In particolare, si assume che, in condizioni normali di preparazione, la temperatura degli pneumatici aumenti in modo progressivo e non attraversi completamente la finestra $[T_{\min}, T_{\max}]$ con un salto diretto tra due giri consecutivi. Sotto questa ipotesi, e per l'ipotesi 6, secondo cui esiste sempre un giro la cui mediana appartiene alla finestra, proseguendo nella successione dei giri di preparazione si raggiunge prima o poi una condizione per cui: $T_{\min} \leq m_j \leq T_{\max}$ e l'insieme su cui è definito il minimo nella definizione di j^* risulta quindi non vuoto.

Si tratta tuttavia di una condizione valida unicamente nello scenario ideale qui considerato, e non di una garanzia universale. In un contesto reale, infatti, la successione dei giri di preparazione potrebbe interrompersi prima di raggiungere la finestra desiderata, per cause esterne al modello, ad esempio: un incidente che comporti il ritiro della vettura, oppure l'esposizione di una bandiera gialla, di una bandiera rossa o della safety car, che interrompa o concluda anticipatamente la sessione di qualifica. In tali casi la successione dei giri di preparazione termina prima che la condizione richiesta venga soddisfatta, e il giro j^* non esiste.

5.5 Possibile estensione a quattro pneumatici

Il modello base assume, con l'ipotesi 1, che i quattro pneumatici abbiano lo stesso comportamento termico, rappresentando così il sistema con un'unica sequenza di temperature. Una versione più realistica manterrebbe invece distinti i dati provenienti da ciascuno pneumatico.

Si indica con:

$$w \in \{FL, FR, RL, RR\}$$

la posizione dello pneumatico, dove FL è l'anteriore sinistro, FR l'anteriore destro, RL il posteriore sinistro e RR il posteriore destro. Per ogni giro j si hanno quindi quattro sequenze distinte di campioni, una per ciascuno pneumatico:

$$T_j^w = (t_{j,1}^w, t_{j,2}^w, \ldots, t_{j,N_j^w}^w) \qquad w \in \{FL, FR, RL, RR\}$$

Applicando l'algoritmo multi-pass separatamente alle quattro sequenze si ottiene, per ogni giro, una mediana per ciascuno pneumatico: m_j^{FL}, m_j^{FR}, m_j^{RL}, m_j^{RR}. Il problema di selezione resta dunque invariato, e va semplicemente risolto quattro volte anziché una. Si pone però un problema applicativo, come combinare le quattro mediane in un'unica decisione Push o Prep.

Possibili strategie decisionali. Il criterio più conservativo richiede che tutti e quattro gli pneumatici si trovino contemporaneamente nella finestra termica ottimale, e comunica Push solo se:

$$T_{\min} \leq m_j^w \leq T_{\max} \qquad \forall w \in \{FL, FR, RL, RR\}$$

È sufficiente che una sola delle quattro mediane cada al di fuori della finestra perché il sistema comunichi Prep. Il criterio evita quindi che una condizione sfavorevole su un singolo pneumatico venga compensata dagli altri tre, risultando però più restrittivo.

Un'alternativa più permissiva consiste nel ricondurre le quattro mediane a un unico valore, prendendone la media:

$$\overline{m}_j = \frac{m_j^{FL} + m_j^{FR} + m_j^{RL} + m_j^{RR}}{4}$$

e comunicare Push quando $T_{\min} \leq \overline{m}_j \leq T_{\max}$. Il limite di questa strategia è evidente, infatti temperature molto diverse tra loro possono compensarsi a vicenda, per cui uno pneumatico troppo caldo e uno troppo freddo potrebbero restituire una media interna alla finestra anche se nessuno dei due si trova realmente nella condizione desiderata.

Una terza via consiste nel pesare diversamente i quattro pneumatici in base alle caratteristiche del circuito e ai carichi a cui sono sottoposti, e dare una maggiore priorità a certe ruote piuttosto che altre. Introducendo dei pesi per ciascuna posizione $\alpha_{FL}, \alpha_{FR}, \alpha_{RL}, \alpha_{RR}$ tali che:

$$\alpha_{FL} + \alpha_{FR} + \alpha_{RL} + \alpha_{RR} = 1$$

si può calcolare una temperatura totale pesata:

$$\widetilde{m}_j = \alpha_{FL} m_j^{FL} + \alpha_{FR} m_j^{FR} + \alpha_{RL} m_j^{RL} + \alpha_{RR} m_j^{RR}$$

e comunicare PUSH quando $T_{\min} \leq \widetilde{m}_j \leq T_{\max}$, attribuendo peso maggiore agli pneumatici che, per la morfologia del circuito e per il tipo di carico dinamico atteso, risultano più critici.

Di queste tre strategie, l'ultima è quella che più si avvicina alla gestione reale degli pneumatici, anche se nella pratica non si traduce in una semplice media pesata: i due assi vengono monitorati separatamente, cercando un equilibrio termico tra anteriore e posteriore anche in funzione del setup della monoposto e dei carichi sviluppati in frenata, percorrenza e trazione [5].

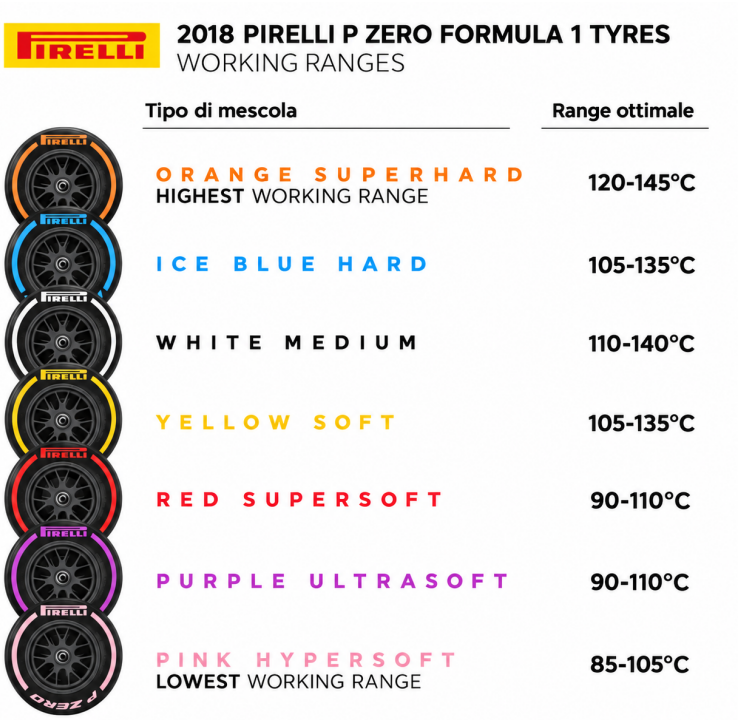


Figura 5.2: Finestre di funzionamento delle mescole Pirelli impiegate in Formula 1 nel 2018 [6, 7].

La Figura 5.2 illustra un'ulteriore semplificazione del modello, i valori $T_{\min}$ e $T_{\max}$ dipendono infatti dalla mescola utilizzata, e in un'applicazione reale andrebbero fissati in base allo pneumatico specifico. Non a caso, per un giro di qualifica la scelta ricade quasi sempre sulla mescola *hypersoft*, il cui *working range* più basso permette di raggiungere la finestra termica ottimale in tempi più brevi.

Osservazione 5.4 (Differenze tra i quattro pneumatici)**.** Nello scenario reale i quattro pneumatici non seguono in generale la stessa evoluzione termica. Le differenze derivano innanzitutto dalla geometria del circuito, curve percorse prevalentemente in una direzione caricano alcuni pneumatici più di altri, mentre frenate e trazione sollecitano in modo diverso l'asse anteriore e quello posteriore.

Anche lo stile di guida introduce ulteriori differenze: un bloccaggio in frenata interessa uno specifico pneumatico e può produrre un *flat spot* o un'usura localizzata, così come uscite di traiettoria o passaggi sulla ghiaia ne alterano in modo asimmetrico l'evoluzione termica.

Per questo, nella pratica, l'obiettivo non è una temperatura media della vettura, ma un equilibrio termico tra i singoli pneumatici e tra i due assi. L'ipotesi 1 resta quindi una semplificazione adottata per mantenere compatto il modello base.

Bibliografia

[1] M. Blum, R. W. Floyd, V. Pratt, R. L. Rivest e R. E. Tarjan, *Time Bounds for Selection*, Journal of Computer and System Sciences, vol. 7, n. 4, pp. 448–461, 1973. https://doi.org/10.1016/S0022-0000(73)80033-9.

[2] T. H. Cormen, C. E. Leiserson, R. L. Rivest e C. Stein, *Introduction to Algorithms*, 3ª edizione, The MIT Press, 2009 (in particolare il capitolo sulla selezione). https://mitpress.mit.edu/9780262033848/introduction-to-algorithms/.

[3] J. I. Munro e M. S. Paterson, *Selection and Sorting with Limited Storage*, Theoretical Computer Science, vol. 12, n. 3, pp. 315–323, 1980. https://doi.org/10.1016/0304-3975(80)90061-4.

[4] C. Renzi, *Algoritmi Sequenziali e di Streaming per la Selezione della Mediana*, report, 26 giugno 2026, materiale fornito per consultazione.

[5] Pirelli, *Seeing Through the Window*, 5 giugno 2017. https://www.pirelli.com/global/en-ww/race/racingspot/seeing-through-the-window-50931/.

[6] Pirelli, *Pirelli Reveals Its Rainbow Tyre Range for Formula 1*, 2017 (presentazione della gamma 2018). https://www.pirelli.com/global/en-ww/race/racingspot/formula-1/pirelli-reveals-its-rainbow-tyre-range-for-formula-1-51271/.

[7] Motorsport.com, *Pirelli Could Widen Working Range of Tyres for 2020*, 2019. Contiene anche un confronto con gli intervalli di funzionamento delle mescole 2018. https://www.motorsport.com/f1/news/pirelli-widen-working-range-2020/4448199/.

[8] Formula 1, *Formula 1 Pirelli Gran Premio d'Italia 2022 – Qualifying*, risultati ufficiali delle qualifiche del 10 settembre 2022. https://www.formula1.com/en/results/2022/races/1120/italy/qualifying.

[9] Fédération Internationale de l'Automobile (FIA), *Formula 1 Pirelli Gran Premio d'Italia 2022 – Qualifying Session Best Sector Times*, 10 settembre 2022. https://www.fia.com/sites/default/files/2022_16_ita_f1_q0_timing_qualifyingsessionbestsectortimes_v01.pdf.

[10] Fédération Internationale de l'Automobile (FIA), *2022 Monza Event – Circuit Map*, documento n. 1, 8 settembre 2022. https://www.fia.com/sites/default/files/doc_1_-_2022_monza_event_-_circuit_map_-_monza_2022.pdf.

[11] Python Software Foundation, *Python Documentation*, documentazione ufficiale. https://docs.python.org/3/.

[12] Matplotlib Development Team, *Matplotlib: Visualization with Python*, documentazione ufficiale. https://matplotlib.org/stable/.